A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security

by

Monowar Hossen Sourav (221002578)
Md. Sajeeb Mia (221002251)
Md. Naim Ahammed (221002599)

Thesis (CSE 400) submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Under the supervision of

Jannathul Moawa Hasi

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GREEN UNIVERSITY OF BANGLADESH

SPRING 2026

© *Monowar Hossen Sourav*, *Md. Sajeeb Mia*, and *Md. Naim Ahammed*
All rights reserved

DECLARATION

Title	A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security
Authors	*Monowar Hossen Sourav*, *Md. Sajeeb Mia*, and *Md. Naim Ahammed*
Student IDs	221002578, 221002251, and 221002599
Supervisor	Jannathul Moawa Hasi

We declare that this thesis entitled *A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security* is the result of our own work except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Monowar Hossen Sourav
221002578

Department of Computer Science and Engineering
Green University of Bangladesh

Md. Sajeeb Mia
221002251

Department of Computer Science and Engineering
Green University of Bangladesh

Md. Naim Ahammed
221002599

Department of Computer Science and Engineering
Green University of Bangladesh

Date: May 2026

CERTIFICATE OF ENDORSEMENT

This is to certify that the thesis entitled *A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security* by **Monowar Hossen Sourav** (Student ID: 221002578), **Md. Sajeeb Mia** (Student ID: 221002251) and **Md. Naim Ahammed** (Student ID: 221002599) has been successfully defended and examined.

Based on the evaluation and discussion held on **May 2026**, we, the undersigned, hereby endorse the thesis for acceptance and approval.

External Member's Name
External Member, Board: 1

External Member's Position
External Member's Affiliation

Board Member's Name
Board Member, Board: 1

Board Member's Position
Department of Computer Science and Engineering
Green University of Bangladesh

Board Chair's Name
Chair, Board: 1

Board Chair's Position
Department of Computer Science and Engineering
Green University of Bangladesh

Date: May 2026

Department of Computer Science and Engineering
Green University of Bangladesh
Kanchon, Rupganj, Narayanganj - 1461

CERTIFICATE

This is to certify that the thesis entitled **A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security**, submitted by **Monowar Hossen Sourav** (Student ID: 221002578), **Md. Sajeeb Mia** (Student ID: 221002251) and **Md. Naim Ahammed** (Student ID: 221002599) are undergraduate students of the **Department of Computer Science and Engineering** has been examined. Upon recommendation by the examination committee, we hereby accord our approval of it as the presented work and submitted report fulfill the requirements for its acceptance in partial fulfillment for the degree of *Bachelor of Science in Computer Science and Engineering*.

Jannathul Moawa Hasi
Lecturer

Department of Computer Science and Engineering
Green University of Bangladesh

Name of the Chairperson
Chairperson

Department of Computer Science and Engineering
Green University of Bangladesh

Place: Narayanganj
Date: May 2026

ACKNOWLEDGMENT

We would like to express our deep and sincere gratitude to our research supervisor, *Jannathul Moawa Hasi*, for giving us the opportunity to conduct research and providing invaluable guidance throughout this work. Her dynamism, vision, sincerity and motivation have deeply inspired us. She has taught us the methodology to carry out the work and to present the works as clearly as possible. It was a great privilege and honor to work and study under her guidance.

We are greatly indebted to our honorable teachers of the Department of Computer Science and Engineering at the Green University of Bangladesh who taught us during the course of our study. Without any doubt, their teaching and guidance have completely transformed us to the persons that we are today.

We are extremely thankful to our parents for their unconditional love, endless prayers and caring, and immense sacrifices for educating and preparing us for our future. We would like to say thanks to our friends and relatives for their kind support and care.

Finally, we would like to thank all the people who have supported us to complete the project work directly or indirectly.

Monowar Hossen Sourav, *Md. Sajeeb Mia* and *Md. Naim Ahammed*
Green University of Bangladesh
Date: May 2026

Dedicated to Almighty Allah —
All praise to Him for His endless mercy and guidance. Dedicated to my beloved parents for their love and support, and to all who inspired and encouraged me along the way.

– Monowar Hossen Sourav

Dedicated to Almighty Allah —
All praise to Him for His endless mercy and guidance. Dedicated to my beloved parents for their love and support.

– Md. Sajeeb Mia

Dedicated to Almighty Allah —
All praise to Him for His endless mercy and guidance. Dedicated to my beloved parents for their love and support.

– Md. Naim Ahammed

ABSTRACT

Electronic voting systems face core security challenges in simultaneously providing ballot privacy, vote integrity, voter anonymity, coercion resistance, and verifiability. Existing blockchain-based e-voting solutions address only subsets of these requirements, leaving critical gaps in coercion resistance and post-quantum security. This thesis presents a blockchain-based e-voting framework that integrates seven cryptographic protocols into a unified voting pipeline: Paillier homomorphic encryption for encrypted tallying, Pedersen commitments with zero-knowledge Σ-proofs for vote validity verification, linkable ring signatures for anonymous ballot submission, Self-Masking Deniable Credentials (SMDC) for coercion resistance, Samplable Anonymous Aggregation (SA^2) for two-server vote privacy, Kyber768 for post-quantum hybrid encryption, and Damgård-Jurik-Nielsen threshold decryption for distributed key management. The system is implemented in Go and deployed on Hyperledger Fabric v2.5 [1]. Performance evaluation confirms an $O(n \cdot m)$ total tally complexity ($O(n)$ per candidate) with a constant per-vote per-candidate cost of approximately 6.1 μs, verified empirically from 100 to 100,000 voters, a complete pipeline time of 70.1 ms per vote, and blockchain throughput of approximately 200 TPS with 100% success rate at 10,000 transactions. Compared with prior work, the proposed framework reduces the asymptotic tally cost from $O(n \cdot m^2)$ (ISE-Voting) to $O(n \cdot m)$ — a factor-m improvement — replaces BP-Vot's $\geq$ 98% differential-privacy approximation with 100% exact integer tallies, and adds post-quantum protection at only 5.35 ms per vote ($\approx$ 7.6% of the pipeline) via the Kyber768 hybrid encryption layer. The novel SMDC mechanism provides computationally indistinguishable real and fake credentials with $k = 5$ slots, enabling coercion resistance in practical blockchain voting contexts. Comparative analysis shows that the proposed framework is the first system to integrate all seven security properties with empirical performance validation on production blockchain infrastructure.

Keywords: blockchain, e-voting, homomorphic encryption, zero-knowledge proofs, ring signatures, coercion resistance, post-quantum cryptography, Hyperledger Fabric, deniable credentials, anonymous aggregation

Contents

List of Acronyms **1**

1 Introduction **2**
1.1 Background . . . 2
1.1.1 Electronic Voting Systems . . . 2
1.1.1.1 Evolution of electronic voting . . . 2
1.1.1.2 E-voting system architecture . . . 3
1.1.2 Security Challenges in E-Voting . . . 4
1.1.2.1 Traditional security approaches . . . 4
1.1.2.2 Cryptographic approaches . . . 4
1.1.3 Traditional E-Voting vs. Blockchain-Based E-Voting . . . 5
1.1.3.1 Core differences . . . 5
1.1.3.2 Advantages of blockchain-based approach . . . 5
1.1.4 Blockchain Technology in E-Voting . . . 5
1.1.5 Cryptographic Protocols for E-Voting . . . 6
1.1.6 State of the Art in the Modern Context . . . 7
1.1.7 Importance and Relevance of the Topic . . . 7
1.2 Motivation . . . 8
1.3 Scope and Limitations . . . 9
1.3.1 In Scope . . . 9
1.3.2 Out of Scope . . . 9
1.4 Problem Statement . . . 10
1.5 Research Objectives . . . 10
1.6 Research Contributions . . . 11
1.7 Thesis Outline . . . 12
1.8 Conclusion . . . 13

2 Related Works **14**
2.1 Introduction . . . 14
2.2 Review of Selected Works . . . 14
2.2.1 Improved Secure and Efficient E-Voting on Blockchain . . . 14
2.2.1.1 Methodology . . . 14

2.2.1.2 Contributions . . . 15
2.2.1.3 Limitations . . . 15
2.2.2 Blockchain-Based E-Voting with Differential Privacy . . . 15
2.2.2.1 Methodology . . . 16
2.2.2.2 Contributions . . . 16
2.2.2.3 Limitations . . . 16
2.2.3 Biometric-Authenticated Blockchain Voting . . . 17
2.2.3.1 Methodology . . . 17
2.2.3.2 Contributions . . . 17
2.2.3.3 Limitations . . . 17
2.2.4 Timed-Release E-Voting with Paillier Homomorphic Encryption . . . 18
2.2.4.1 Methodology . . . 18
2.2.4.2 Contributions . . . 18
2.2.4.3 Limitations . . . 18
2.2.5 Scalable Coercion-Resistant Blockchain Voting . . . 19
2.2.5.1 Methodology . . . 19
2.2.5.2 Contributions . . . 19
2.2.5.3 Limitations . . . 19
2.2.6 Survey: Blockchain for Securing Electronic Voting Systems . . . 20
2.2.6.1 Methodology . . . 20
2.2.6.2 Contributions . . . 20
2.2.6.3 Limitations . . . 20
2.3 Background Technology . . . 21
2.3.1 Paillier Public-Key Cryptosystem . . . 21
2.3.2 Pedersen Commitments . . . 21
2.3.3 Zero-Knowledge Σ-Protocols . . . 21
2.3.4 Linkable Ring Signatures . . . 22
2.3.5 Threshold Cryptography and Secret Sharing . . . 22
2.3.6 Cryptographic Hash Functions and Message Authentication . . . 22
2.3.7 Post-Quantum Key Encapsulation: ML-KEM (Kyber) . . . 22
2.3.8 Authenticated Symmetric Encryption: AES-GCM . . . 23
2.3.9 Hyperledger Fabric . . . 23
2.3.10 Samplable Anonymous Aggregation . . . 23
2.4 Comparison Among State-of-the-Art Works . . . 23
2.5 Summary . . . 25

3 System Design and Cryptographic Protocols **27**
3.1 Introduction . . . 27
3.2 Proposed Framework and Assumptions . . . 27
3.3 Notation . . . 29
3.4 Cryptographic Protocol Formulation . . . 30
3.4.1 Paillier Homomorphic Encryption . . . 30
3.4.1.1 Key Generation . . . 30
3.4.1.2 Encryption . . . 30
3.4.1.3 Decryption . . . 31
3.4.1.4 Homomorphic Properties . . . 31
3.4.2 Pedersen Commitments . . . 31
3.4.2.1 Parameter Generation . . . 31
3.4.2.2 Commitment . . . 31
3.4.3 Zero-Knowledge Proofs (Σ-Protocol) . . . 32
3.4.3.1 Binary Proof . . . 32
3.4.3.2 Sum Proof . . . 33
3.4.4 Linkable Ring Signatures . . . 33
3.4.4.1 Key Generation . . . 33
3.4.4.2 Signing . . . 33
3.4.4.3 Verification and Linkability . . . 34
3.4.5 SMDC: Self-Masking Deniable Credentials . . . 34
3.4.5.1 Credential Generation . . . 34
3.4.5.2 Coercion Resistance Property . . . 35
3.4.5.3 Credential Verification . . . 36
3.4.6 SA^2: Samplable Anonymous Aggregation . . . 36
3.4.6.1 Vote Splitting . . . 36
3.4.6.2 Reconstruction . . . 36
3.4.7 Kyber768 Post-Quantum Hybrid Encryption . . . 37
3.4.7.1 Hardness Assumption and Parameter Selection . . . 37
3.4.7.2 KEM-DEM Hybrid Construction . . . 37
3.4.7.3 Scope and Limitations of Post-Quantum Coverage . . . 38
3.4.8 Threshold Paillier Decryption . . . 38
3.4.8.1 Key Distribution . . . 38
3.4.8.2 Partial Decryption . . . 38
3.4.8.3 Combination . . . 39
3.5 Vote Casting Pipeline . . . 39
3.6 Blockchain Integration . . . 43
3.7 Engineering and Societal Considerations . . . 44
3.7.1 Societal Impact . . . 44
3.7.2 Sustainability . . . 44

3.7.3 Accessibility . . . 44
3.7.4 Ethical Considerations . . . 44
3.8 Security Analysis . . . 44
3.8.1 Formal Security Theorems . . . 45
3.8.2 Duress Detection . . . 46
3.9 Conclusion . . . 47

4 Implementation, Results and Discussion **49**
4.1 Introduction . . . 49
4.2 Implementation Environment . . . 49
4.3 Performance Metrics . . . 50
4.4 Experimental Results . . . 51
4.4.1 Cryptographic Micro-Benchmarks . . . 51
4.4.1.1 Post-Quantum Layer Cost Analysis . . . 52
4.4.2 End-to-End Pipeline Benchmark . . . 53
4.4.3 Scalability Validation . . . 54
4.4.3.1 Homomorphic Tally Scalability . . . 54
4.4.3.2 Linear-in-m Total Scaling (Candidate-Count Validation) . . . 55
4.4.3.3 Ring Signature Scalability . . . 56
4.4.4 Blockchain Performance . . . 57
4.5 Discussion . . . 58
4.5.1 Pipeline Bottleneck Analysis . . . 59
4.5.2 Scalability Significance . . . 59
4.5.3 Blockchain Performance Analysis . . . 60
4.5.4 Practical Deployment Considerations . . . 61
4.6 Test Coverage Analysis . . . 61
4.7 Formal Verification with ProVerif . . . 62
4.8 Comparative Analysis . . . 62
4.8.1 Comparison With Related Works . . . 62

5 Conclusion and Future Work **65**
5.1 Introduction . . . 65
5.2 Summary of the Study . . . 65
5.3 Limitations and Future Work . . . 66

References **69**

A ProVerif Formal Verification Artefacts **73**
A.1 Reproducing the Verification 73
A.2 Ballot Privacy: `privacy.pv` 73
A.3 Individual Verifiability: `verifiability.pv` 75
A.4 Voter Eligibility: `eligibility.pv` 77
A.5 Summary 78

List of Figures

3.1 Proposed System Architecture Overview 28
3.2 SMDC Deniable Credential Slot Structure 35
3.3 Seventeen-Step Vote Casting Pipeline 40
3.4 Voter Registration and Casting Activity 41
3.5 Vote Casting Sequence Diagram 42

4.1 Cryptographic Primitive Per-Operation Cost 52
4.2 Vote Pipeline Per-Phase Breakdown 53
4.3 Homomorphic Tally Scaling with Voter Count 55
4.4 Tally Time versus Candidate Count 56
4.5 Ring Signature Sign and Verify Scaling 57
4.6 Blockchain Throughput and Latency Benchmark 58

List of Tables

2.1 Comparison of State-of-the-Art Blockchain E-Voting Approaches 24

3.1 Mathematical Notation 29

4.1 Implementation Environment 50
4.2 Cryptographic Micro-Benchmark Results 51
4.3 Vote Casting Pipeline: Per-Phase Breakdown 53
4.4 Homomorphic Tally Scalability Results 54
4.5 Candidate-Independent Tally Scaling ($n = 1{,}000$) 55
4.6 Ring Signature Scalability 56
4.7 Hyperledger Caliper Blockchain Benchmark Results 57
4.8 Test Coverage by Package 61
4.9 ProVerif Formal Verification Results 62
4.10 Comparison of the Proposed Method with Existing Works 63

List of Acronyms

AES	Advanced Encryption Standard
CAI	Cast-As-Intended
CSP	Cloud Service Provider
DCRA	Decisional Composite Residuosity Assumption
DJN	Damgård-Jurik-Nielsen
DRE	Direct Recording Electronic
GCM	Galois/Counter Mode
gRPC	Google Remote Procedure Call
HE	Homomorphic Encryption
HMAC	Hash-based Message Authentication Code
IoT	Internet of Things
JCJ	Juels-Catalano-Jakobsson
LWE	Learning With Errors
ML-KEM	Module-Lattice Key Encapsulation Mechanism
MSP	Membership Service Provider
NIST	National Institute of Standards and Technology
NIZK	Non-Interactive Zero-Knowledge
PQ	Post-Quantum
RSA	Rivest-Shamir-Adleman
SA^2	Samplable Anonymous Aggregation
SHA	Secure Hash Algorithm
SMDC	Self-Masking Deniable Credentials
SSI	Self-Sovereign Identity
TLS	Transport Layer Security
TPS	Transactions Per Second
TRE	Timed-Release Encryption
ZK	Zero-Knowledge
ZKP	Zero-Knowledge Proof

1. Introduction

1.1 Background

Electronic voting (e-voting) has emerged as a practical approach to modernizing democratic processes by replacing traditional paper-based balloting with digital systems. As nations worldwide seek to improve voter accessibility, reduce administrative costs, and accelerate result tabulation, e-voting systems have gained considerable attention from both the research community and government institutions. However, adopting electronic voting introduces core security challenges that do not exist in conventional voting: ballot privacy, vote integrity, coercion resistance, and universal verifiability must all be simultaneously guaranteed in a system where digital data can be copied, intercepted, or manipulated.

The integration of blockchain technology [2] with advanced cryptographic protocols offers a promising foundation for addressing these challenges. By leveraging the immutability and transparency properties of distributed ledgers alongside homomorphic encryption, zero-knowledge proofs, and anonymous credential schemes, it becomes possible to construct voting systems that provide mathematical guarantees of security rather than relying solely on organizational trust.

1.1.1 Electronic Voting Systems

Electronic voting encompasses any voting method where the voter's intent is recorded through electronic means. These systems range from Direct Recording Electronic (DRE) machines used at physical polling stations to internet-based remote voting platforms that allow participation from any location.

1.1.1.1 Evolution of electronic voting

The evolution of electronic voting can be traced through several distinct generations. First-generation systems, deployed in the 1990s and early 2000s, were primarily DRE machines that digitized the ballot-marking process but stored votes in centralized databases controlled by election authorities. These systems improved tallying speed but introduced single points of failure and required voters to trust the machine vendor and election administrators completely.

Second-generation systems introduced cryptographic techniques such as mix-nets and blind signatures to provide some degree of voter anonymity. Notable examples include the Helios voting system, which pioneered web-based verifiable elections for organizational contexts. However, these systems relied on trusted third parties for key management and ballot processing, leaving them vulnerable to insider attacks and coercion.

Third-generation systems, emerging from 2018 onward, began integrating blockchain technology to provide immutable vote records and decentralized trust. Systems such as Voatz (used in US military overseas voting) and Agora (deployed as an observer in Sierra Leone's 2018 elections) represented early attempts at blockchain-based voting. However, these systems were criticized for weak cryptographic guarantees; a 2020 MIT security analysis of Voatz [3] revealed critical vulnerabilities including client-side attacks, server compromise possibilities, and vote manipulation risks.

Current e-voting research focuses on combining multiple cryptographic primitives into multi-layered security frameworks. The proposed framework represents this generation by integrating seven distinct cryptographic protocols with a production blockchain infrastructure to simultaneously address privacy, integrity, coercion resistance, and post-quantum security.

1.1.1.2 E-voting system architecture

A modern e-voting system architecture typically consists of four functional layers that interact to ensure secure vote processing:

1. **Authentication Layer**: Responsible for verifying voter identity and eligibility. This may involve biometric verification, digital certificates, or credential-based authentication. The authentication mechanism must confirm the voter's right to participate without revealing their identity to other system components.

2. **Computation Layer**: Handles the cryptographic operations that protect ballot privacy and ensure vote validity. This includes encryption, commitment generation, zero-knowledge proof construction, and digital signature creation. The computational demands of this layer directly impact the per-vote processing time and overall system throughput.

3. **Privacy Layer**: Ensures that individual votes cannot be linked to specific voters even by system administrators. Privacy-preserving techniques include mix networks, homomorphic encryption, secret sharing, and anonymous aggregation protocols. The strength of this layer determines whether the system can resist surveillance and coercion attacks.

4. **Storage Layer**: Provides persistent, tamper-evident storage of vote records. Blockchain technology serves this function by creating an append-only ledger that is distributed

across multiple nodes, making retrospective vote manipulation computationally infeasible.

1.1.2 Security Challenges in E-Voting

The security requirements of electronic voting are considerably more demanding than those of typical information systems. A voting system must simultaneously satisfy properties that are often in tension with one another; for example, votes must be both secret (unlinked to voters) and verifiable (provably counted correctly).

1.1.2.1 Traditional security approaches

Traditional approaches to e-voting security relied primarily on organizational controls and access restrictions. Votes were encrypted using standard symmetric or asymmetric algorithms (e.g., AES-256 [4], RSA [5]), and system integrity depended on the trustworthiness of election authorities who held the decryption keys. This model suffers from several critical weaknesses: a compromised authority can decrypt and read individual votes, the tallying process is opaque to outside observers, and there is no mechanism to prove that votes were counted correctly without revealing them.

Beyond the tallying problem, traditional systems provide no protection against voter coercion. If a coercer can observe a voter's actions or demand proof of how they voted, the voter has no way to cast a free vote while appearing to comply with the coercer's demands. This vulnerability is particularly acute in remote voting scenarios where the voter is not protected by the secrecy of a polling booth.

1.1.2.2 Cryptographic approaches

Modern cryptographic approaches address these limitations through mathematical guarantees rather than organizational trust. Homomorphic encryption enables vote tallying without decrypting individual ballots. Zero-knowledge proofs allow voters to prove their votes are well-formed without revealing their content. Ring signatures provide anonymity within a group of potential signers while enabling double-vote detection through key images.

However, achieving all security properties simultaneously through cryptographic means requires careful protocol composition. Individual protocols provide specific guarantees, but their combination must be analyzed to ensure that the security properties are preserved under composition. This challenge of secure multi-protocol integration is a central concern of the proposed system design.

1.1.3 Traditional E-Voting vs. Blockchain-Based E-Voting

The key distinction between traditional and blockchain-based e-voting lies in the trust model. Traditional systems require voters to trust a centralized authority, while blockchain-based systems distribute trust across a network of independent nodes.

1.1.3.1 Core differences

In traditional e-voting, a central election authority controls the encryption keys, manages the vote database, and performs the tally. Voters must trust that this authority will not manipulate votes, leak individual ballots, or selectively exclude certain votes from the count. The verification process, if any, is typically limited to auditing by authorized observers who must again be trusted.

Blockchain-based e-voting fundamentally alters this trust model. Votes are recorded on a distributed ledger that is maintained by multiple independent parties. The immutability property of the blockchain ensures that once a vote is recorded, it cannot be modified or deleted without detection. Smart contracts can enforce voting rules automatically, removing the need to trust election officials to follow procedures correctly. Cryptographic proofs attached to each vote enable universal verifiability: any observer can verify that all recorded votes are well-formed and correctly tallied.

1.1.3.2 Advantages of blockchain-based approach

Several advantages of blockchain-based e-voting are worth highlighting. First, the decentralized nature of the blockchain eliminates single points of failure: no single entity can compromise the entire election. Second, the immutable ledger provides an auditable trail that persists indefinitely, enabling post-election verification at any time. Third, smart contracts ensure consistent enforcement of election rules without human intervention. Fourth, blockchain transparency enables end-to-end verifiability, where voters can verify that their votes were recorded and counted correctly.

These properties are particularly relevant in contexts where institutional trust is low, such as elections in developing nations, organizational governance, or scenarios where election manipulation is a concern.

1.1.4 Blockchain Technology in E-Voting

Blockchain platforms used for e-voting fall into two broad categories: public blockchains (e.g., Ethereum [6]) and permissioned blockchains (e.g., Hyperledger Fabric [1]). Public blockchains offer maximum decentralization but suffer from scalability limitations, high transaction costs, and energy consumption concerns. Permissioned blockchains provide controlled access, higher throughput, and configurable privacy, making them more suitable

for election systems where the set of validating nodes is known and controlled by election authorities.

Our framework employs Hyperledger Fabric v2.5 [1], a permissioned blockchain framework maintained by the Linux Foundation. Fabric's modular architecture supports pluggable consensus mechanisms, channel-based data isolation, and chaincode (smart contracts) written in general-purpose languages such as Go. The Fabric Gateway SDK enables application-level interaction with the blockchain over gRPC channels secured with TLS, providing authenticated communication between the voting application and the distributed ledger. Recent empirical studies [7] have demonstrated that Fabric achieves substantially higher throughput and lower latency than public blockchain alternatives for enterprise applications.

1.1.5 Cryptographic Protocols for E-Voting

The security of the proposed framework rests on seven cryptographic protocols, each providing a specific security guarantee:

- **Paillier Homomorphic Encryption** [8]: Enables encrypted vote tallying without decryption. Based on the Decisional Composite Residuosity Assumption (DCRA) [8], Paillier's additive homomorphism allows the election result to be computed directly from encrypted ballots: $E(a) \times E(b) = E(a + b)$.

- **Pedersen Commitments**: Provide computationally hiding and perfectly binding commitments to vote values. Used as the foundation for zero-knowledge proofs that verify vote validity.

- **Zero-Knowledge Proofs (Σ-Protocol)**: Enable voters to prove that their encrypted votes contain valid values ($w \in \{0, 1\}$) and sum to exactly one ($\sum w = 1$) without revealing which candidate received the vote. The Strong Fiat-Shamir transformation is used to make the proofs non-interactive, with public parameters included in the hash to prevent the Helios vulnerability identified by Bernhard, Pereira, and Warinschi [9].

- **Linkable Ring Signatures**: Provide voter anonymity within a ring of 100 members while enabling double-vote detection through deterministic key images. Based on the Liu-Wei-Wong construction [10].

- **SMDC (Self-Masking Deniable Credentials)**: A novel coercion resistance mechanism where each voter receives $k = 5$ credential slots (1 real + 4 fake). Under coercion, voters can present fake credentials that produce valid-looking but silently discarded votes. The real and fake credentials are computationally indistinguishable due to Pedersen's hiding property.

- **SA^2 (Samplable Anonymous Aggregation)**: A two-server privacy model adapted from Apple's federated learning research [11]. Votes are split into masked shares distributed between two non-colluding servers. As long as one server remains honest, individual votes cannot be reconstructed.

- **Kyber768 Post-Quantum Encryption**: NIST Level 3 post-quantum key encapsulation based on Module-LWE [12], providing protection against future quantum computer attacks through hybrid encryption with Paillier.

1.1.6 State of the Art in the Modern Context

Recent blockchain e-voting research has produced systems that each strengthen one or two security properties but none that simultaneously achieve all of: homomorphic tallying for exact results, formal zero-knowledge proofs for vote validity, linkable ring signatures for anonymity with double-vote detection, deniable credentials for coercion resistance, two-server anonymous aggregation, post-quantum security, and production-grade blockchain deployment with empirical performance data. Recurring shortcomings across the surveyed work include super-linear tally complexity, reliance on trusted counting authorities or differential-privacy approximations, weak symmetric-only privacy, and the absence of a coercion-resistance mechanism that survives a coercer who can demand proof of how the voter cast a ballot. A detailed analysis of representative works, their reported performance, and the precise gaps they leave open is presented in Chapter 2. The proposed framework directly addresses these gaps and is positioned in detail against the cited systems in Section 4.8 of Chapter 4.

1.1.7 Importance and Relevance of the Topic

Secure electronic voting matters beyond technical innovation; it touches on democratic principles directly. In many developing nations, voter intimidation and coercion remain serious obstacles to free elections. International election observers frequently report incidents of vote buying, family voting, and employer pressure on voting choices. A voting system that provides mathematical guarantees of coercion resistance, not just procedural safeguards, would mark a genuine step forward for democratic integrity.

On a related note, the emergence of quantum computing poses a long-term threat to current cryptographic systems [13]. Elections may produce records that must remain confidential for decades. A vote encrypted today with RSA, ECC, or any scheme whose security reduces to integer factorisation or discrete logarithms could potentially be decrypted by a future quantum computer running Shor's algorithm [13], retroactively compromising voter privacy. This *harvest-now-decrypt-later* threat model has motivated the U.S. National Institute of Standards and Technology to standardise lattice-based replacements, including ML-KEM (FIPS 203 [12]) for key encapsulation and ML-DSA (FIPS 204 [14]) for digital

signatures, both finalised in 2024. Post-quantum cryptographic protection is therefore not merely a theoretical concern but a practical and standards-aligned necessity for long-term ballot secrecy.

From a scalability perspective, nations with large populations require voting systems whose per-candidate tally cost grows only linearly with the number of voters and whose total tally cost grows no worse than linearly with the number of candidates. The proposed framework achieves $O(n)$ per candidate and $O(n \cdot m)$ total tally complexity, confirmed through empirical benchmarks, which is a factor-m improvement over ISE-Voting's $O(n \cdot m^2)$ and demonstrates feasibility for national-scale deployments.

The intersection of blockchain technology, advanced cryptography, and post-quantum security in the context of electronic voting represents an active and important area of research with direct societal impact.

1.2 Motivation

The development of the proposed framework is motivated by several critical gaps in existing blockchain-based e-voting systems. Despite active research in this field, current solutions exhibit key limitations that prevent their deployment in real-world elections with strong security requirements.

First, existing systems generally provide only a subset of the required security properties. Systems with strong privacy guarantees (e.g., homomorphic encryption) typically lack coercion resistance mechanisms. Conversely, systems that address coercion resistance (e.g., JCJ [15] and Civitas [16]) rely on theoretical credential management schemes that have not been practically implemented in blockchain contexts. No existing system integrates a comprehensive cryptographic stack that addresses privacy, validity, anonymity, coercion resistance, and post-quantum security simultaneously.

Second, the computational complexity of existing tallying mechanisms limits their scalability. ISE-Voting's $O(n \times m^2)$ complexity means that for an election with 1 million voters and 10 candidates, the tally requires approximately 10^8 operations, making it impractical for large constituencies. A linear-time tallying approach is needed for real-world deployment.

Third, most blockchain-based e-voting research provides only theoretical designs or simulations without production-grade blockchain integration. Performance claims are rarely validated against real blockchain networks with empirical throughput and latency measurements. The gap between theoretical protocol design and production system implementation remains largely unaddressed.

Fourth, the threat of quantum computing is largely ignored in current e-voting research. While several post-quantum e-voting proposals exist in theory, very few integrate post-quantum key encapsulation into a working system alongside classical cryptographic protocols.

These gaps motivate the design and implementation of the proposed framework as a comprehensive, production-ready e-voting system that addresses all of the above challenges through a carefully composed seven-protocol cryptographic stack deployed on real blockchain infrastructure.

1.3 Scope and Limitations

This thesis focuses on the cryptographic protocol design, implementation, and performance evaluation of the proposed system. The scope and boundaries of this research are defined as follows:

1.3.1 In Scope

- Design and implementation of seven cryptographic protocols (Paillier homomorphic encryption [8], Pedersen commitments [17], zero-knowledge Σ-proofs with Strong Fiat-Shamir [9], linkable ring signatures [10], the proposed SMDC deniable credentials, two-server SA^2 aggregation [11], and Kyber-768 ML-KEM [12]) in Go 1.24.
- Integration with Hyperledger Fabric v2.5 [1] as a production blockchain infrastructure.
- Empirical performance evaluation including crypto micro-benchmarks, pipeline analysis, scalability testing (100 to 100,000 voters), and Caliper blockchain benchmarks (10,000 to 100,000 transactions).
- Security analysis with formal theorem statements, proof sketches for six security properties, and ProVerif formal verification of three core properties.
- Comparative analysis with five state-of-the-art blockchain e-voting systems.

1.3.2 Out of Scope

The following items fall outside the boundary of the present work: user interface (UI/UX) design and implementation for the voter-facing application, voter registration infrastructure and national identity integration, formal verification using Tamarin or other automated tools beyond the ProVerif symbolic model, multi-constituency deployment with geographic distribution, mobile client implementation and accessibility testing, and real-world election deployment and usability studies with actual voters. These out-of-scope items are discussed as future work directions in Chapter 5.

1.4 Problem Statement

Despite the advances in blockchain-based e-voting, significant challenges remain in building a system that provides comprehensive security guarantees while maintaining practical performance. The core problems addressed by this thesis are as follows:

- How can multiple cryptographic protocols (homomorphic encryption, zero-knowledge proofs, ring signatures, deniable credentials, anonymous aggregation, and post-quantum key encapsulation) be securely composed into a unified voting pipeline that preserves the individual security properties of each protocol under composition?
- How can coercion resistance be practically implemented in a blockchain e-voting context, such that a coerced voter can present plausible fake credentials to a coercer without the coercer being able to distinguish real from fake votes, while ensuring that only real votes are counted in the final tally?
- How can the vote tallying process achieve $O(n)$ computational complexity *per candidate* (linear in the number of voters), with $O(n \cdot m)$ total cost across m candidates, while preserving the exact correctness of results without introducing noise or approximation?
- How can a production-ready blockchain infrastructure (Hyperledger Fabric) be integrated with the cryptographic voting pipeline to provide immutable vote records and chaincode-level verification, and what throughput and latency characteristics does this integration achieve under realistic load conditions?

1.5 Research Objectives

The primary goal of this research is to design, implement, and empirically evaluate a blockchain-based e-voting system that provides comprehensive cryptographic security guarantees while demonstrating practical performance on production infrastructure. The specific objectives are:

- To design and implement a seven-protocol cryptographic voting pipeline that provides ballot privacy (Paillier homomorphic encryption), vote validity (zero-knowledge Σ-proofs), voter anonymity (linkable ring signatures), coercion resistance (SMDC deniable credentials), privacy-preserving aggregation (SA^2 two-server model), and post-quantum security (Kyber768 hybrid encryption).
- To develop a novel coercion resistance mechanism (SMDC) based on deniable credentials with $k = 5$ slots, integrated with behavioral duress detection, that provides computationally indistinguishable real and fake credentials in a practical blockchain voting context.

- To achieve $O(n)$ per-candidate tally complexity (and $O(n \cdot m)$ total) through Paillier's additive homomorphism, enabling linear-in-voters vote counting that grows only linearly — not quadratically — with the number of candidates, and to validate this complexity through empirical benchmarks from 100 to 100,000 voters.
- To integrate the voting system with a production Hyperledger Fabric v2.5 blockchain network and empirically measure throughput and latency using Hyperledger Caliper benchmarks across transaction volumes from 10,000 to 100,000.

1.6 Research Contributions

This thesis makes the following contributions to the field of blockchain-based electronic voting. To avoid overstating novelty, the contributions are explicitly grouped into (a) genuinely novel cryptographic constructions, (b) system-integration and engineering contributions, and (c) empirical results.

1. **SMDC Coercion Resistance (Novel Construction).** The thesis introduces Self-Masking Deniable Credentials (SMDC), a practical coercion resistance mechanism in which each voter receives $k = 5$ credential slots (1 real and 4 fake). The real slot index is derived via HMAC-SHA256 and never stored, making it re-derivable only by the legitimate voter. To the best of our knowledge, this is the first complete implementation of deniable credentials in a blockchain e-voting context.
2. **Behavioral Duress Detection (Novel Construction).** A behavioral coercion detection mechanism is presented, using HMAC-SHA256 hashing of biometric signals (blink count, head tilt, long press, time delay, voice command) with constant-time equality verification against a per-voter stored hash. The deterministic per-voter equality check and the silent-substitution semantics are original to this work.
3. **Coercion Short-Circuit Semantics (Novel Construction).** The vote-casting pipeline is extended with silent-substitution semantics that interact with SMDC fake-slot detection: under duress, the full ring-signature and SA^2 share-split steps still execute (preserving timing indistinguishability), but the key-image registration is silently discarded so that the receipt returned to a coerced voter is observationally identical to a genuine cast.
4. **Seven-Protocol Pipeline Integration (Engineering Contribution).** This work presents the first complete implementation that composes seven cryptographic primitives—Paillier homomorphic encryption [8], Pedersen commitments [17], zero-knowledge Σ-proofs (Strong Fiat-Shamir) [9], linkable ring signatures [10], SA^2 anonymous aggregation [11], Kyber768 post-quantum hybrid encryption [12], and DJN threshold Paillier decryption [18]—together with the novel SMDC and duress mechanisms

above into a single, functional voting pipeline. The underlying primitives are existing constructions; the contribution lies in their composition, the resolution of inter-protocol parameter compatibility, and the production engineering.

5. **SA^2 Adaptation for Voting (Integration Choice, not Novel).** Apple's Samplable Anonymous Aggregation primitive [11], originally designed for federated data analysis, is integrated with Paillier additive homomorphism so that the encrypted tally is reconstructed via mask cancellation across two non-colluding aggregators. The SA^2 construction itself is not a contribution of this thesis; the contribution is its placement in the voting pipeline.

6. **Production Hyperledger Fabric Integration (Engineering Contribution).** The system is fully integrated with Hyperledger Fabric v2.5 [1] via the Fabric Gateway SDK (fabric-gateway v1.10.1), with empirical Hyperledger Caliper benchmarks demonstrating approximately 200 TPS throughput and 100% success rate at 10,000 transactions.

7. **$O(n)$-per-Candidate Tally with Empirical Validation (Result).** Through Paillier's additive homomorphism, the tally achieves $O(n)$ complexity per candidate and $O(n \cdot m)$ total complexity, confirmed empirically: the per-vote per-candidate cost is approximately 6.1 μs across four orders of magnitude (100 to 100,000 voters), and the total tally time scales linearly with m (Section 4.4.3.2). This is a factor-m improvement over ISE-Voting's $O(n \cdot m^2)$, and the 100% exact tally contrasts with BP-Vot's $\geq 98\%$ differential-privacy approximation.

1.7 Thesis Outline

The remainder of this thesis is organized as follows:

- **Chapter 2, Related Works**, discusses the existing research on blockchain-based e-voting systems with a focus on security properties, cryptographic techniques, and coercion resistance mechanisms. It reviews six key works in detail, analyzing their methodologies, contributions, and limitations. The chapter concludes with a comprehensive comparison table that positions our framework relative to the state of the art.

- **Chapter 3, System Design and Cryptographic Protocols**, presents the complete architecture of the proposed system. This chapter includes the detailed mathematical formulation of all seven cryptographic protocols, the 17-step vote casting pipeline, the SMDC credential generation mechanism, the SA^2 aggregation protocol, and the blockchain integration architecture. Engineering and societal considerations are also addressed.

- **Chapter 4, Implementation and Performance Evaluation**, describes the implementation details including the technology stack (Go 1.24, Gin 1.11, Hyperledger Fabric v2.5), test coverage analysis, and comprehensive performance benchmarks. Results include crypto micro-benchmarks, end-to-end pipeline measurements, scalability validation from 100 to 100,000 voters, and Hyperledger Caliper blockchain throughput benchmarks from 10,000 to 100,000 transactions. Comparative analysis with existing systems is presented.

- **Chapter 5, Conclusion and Future Work**, summarizes the key findings of this research, discusses the system's limitations, and outlines directions for future work including formal verification, lattice-based ring signatures, and production deployment considerations.

1.8 Conclusion

This chapter has provided the background, motivation, and problem statement for this research. The fundamental security challenges in electronic voting (ballot privacy, vote integrity, coercion resistance, and post-quantum security) were identified, along with the limitations of existing blockchain-based solutions. The research objectives were defined to address these gaps through a seven-protocol cryptographic stack deployed on production blockchain infrastructure. The next chapter reviews the existing literature in detail, analyzing the methodology, contributions, and limitations of state-of-the-art blockchain e-voting systems.

2. Related Works

2.1 Introduction

Blockchain-based electronic voting has attracted growing research interest in recent years, driven by the demand for transparent, secure, and verifiable election systems. As democratic processes face increasing scrutiny regarding integrity and trust, researchers have explored various combinations of blockchain technology, cryptographic protocols, and privacy-preserving mechanisms to meet the core security requirements of voting.

This chapter reviews six key works that represent the current state of the art in blockchain-based e-voting. The reviewed papers span multiple security dimensions including privacy preservation through homomorphic encryption and differential privacy, voter authentication through biometric verification, coercion resistance through formal game-based proofs, post-quantum security considerations, and broad architectural surveys. Each work is analyzed in terms of its methodology, contributions, and limitations. The chapter concludes with a comparative analysis that positions our framework relative to existing work and identifies the research gaps that this thesis addresses.

2.2 Review of Selected Works

2.2.1 Improved Secure and Efficient E-Voting on Blockchain

The paper by Zhang et al. [19] investigates the problem of achieving voter anonymity, distributed vote counting, and public verifiability in blockchain-based e-voting systems designed for IoT-assisted environments. The authors propose ISE-Voting, a scheme that combines identity-based ring signatures with secret sharing to eliminate the need for a single trusted counting authority.

2.2.1.1 Methodology

The authors formulate the e-voting problem as a multi-party computation task on the Ethereum blockchain. Their approach employs identity-based ring signatures to provide voter anonymity, where each voter signs their ballot using a ring of eligible voter identities. To address the centralized tallying problem, the authors introduce a Cloud Service Provider (CSP) that assists in distributed vote counting through a ballot cutting algorithm. The

voting protocol operates in three phases: registration (where voters obtain identity-based keys from a trusted authority), voting (where ballots are signed and submitted to the blockchain), and counting (where the CSP and blockchain nodes collaboratively tally the results). The secret sharing mechanism distributes the counting responsibility across multiple parties, preventing any single entity from learning individual votes during the tally.

2.2.1.2 Contributions

The main contributions of this work are threefold. First, the authors design an identity-based ring signature scheme that provides 128-bit security for voter anonymity in a claimed post-quantum setting through symmetric key foundations. Second, they propose a distributed counting mechanism using a CSP and ballot cutting algorithm that reduces the computational pressure on blockchain nodes. Third, the authors provide formal security analysis demonstrating correctness, anonymity, unforgeability, and verifiability of the voting protocol. Experimental evaluation shows that ISE-Voting can perform distributed counting within practical timeframes for medium-scale elections.

2.2.1.3 Limitations

Despite its contributions, ISE-Voting exhibits several notable limitations. The tally complexity of the ballot cutting algorithm is $O(n \times m^2)$, where n is the number of voters and m is the number of candidates. For a national-scale election with millions of voters and dozens of candidates, this quadratic dependency on candidate count renders the approach impractical. In addition, the system relies on a CSP as a semi-trusted third party for counting, which partially undermines the decentralization benefits of blockchain. The scheme provides no coercion resistance mechanism: a coerced voter cannot produce a plausible fake vote. Also, while the authors claim post-quantum security through symmetric key foundations, the identity-based signature scheme itself relies on assumptions that may not withstand quantum attacks. The system also lacks homomorphic encryption, meaning individual ballots must be decrypted for counting, creating a privacy vulnerability during the tally phase.

2.2.2 Blockchain-Based E-Voting with Differential Privacy

The paper by Baniata and Caluna [20] addresses the trade-off between voter privacy and election transparency in blockchain-based systems. The authors propose BP-Vot, which integrates smart contracts, differential privacy, and self-sovereign identities on the Hyperledger Besu platform.

2.2.2.1 Methodology

The authors design a permissioned blockchain architecture on Hyperledger Besu where smart contracts govern the election lifecycle including voter registration, ballot submission, and result announcement. The key innovation is a (k, ϵ)-differential privacy mechanism that protects individual voter preferences while allowing statistical approximation of election results. The privacy mechanism works by "pivoting" a randomly selected candidate and redistributing retrievable votes to other candidates, creating plausible deniability for individual choices. Voter authentication is handled through Web3-based self-sovereign identity (SSI), where voters use verifiable credentials stored in their own digital wallets rather than relying on centralized identity providers. The system is deployed on a cloud-based permissioned blockchain with configurable privacy parameters.

2.2.2.2 Contributions

BP-Vot makes several notable contributions. First, it introduces a novel application of differential privacy to blockchain-based voting, achieving voter privacy without requiring homomorphic encryption or mix-nets. Second, the SSI-based authentication provides a decentralized identity management solution that avoids single points of failure. Third, the system demonstrates practical performance with a 24% improvement in latency compared to state-of-the-art solutions (approximately 1 s/TX versus 1.24 s/TX). The authors provide extensive experimental results testing with 10,000 to 50,000 votes and 2 to 8 candidates, demonstrating at least 98% accuracy in vote approximation across all scenarios. Fourth, the differential privacy mechanism is formally evaluated and proven robust against reconstruction attacks.

2.2.2.3 Limitations

The core limitation of BP-Vot lies in its differential privacy approach. By design, differential privacy adds noise to the election results, meaning the published tally is an approximation rather than an exact count. While the authors report $\geq$98% accuracy, this means that in close elections with narrow margins, the system could potentially report incorrect winners. This trade-off between privacy and accuracy is unacceptable in high-stakes political elections where every vote must be counted exactly. Beyond this, BP-Vot provides no coercion resistance mechanism, no zero-knowledge proofs for vote validity verification, and no post-quantum security considerations. The system also lacks anonymous signatures, meaning voter identity is protected only through the differential privacy layer rather than through cryptographic anonymity guarantees.

2.2.3 Biometric-Authenticated Blockchain Voting

Faruk et al. [21] present a blockchain-based voting system that combines biometric authentication with Hyperledger Fabric to address voter impersonation and fraud in online elections. The system was tested with 100 real participants.

2.2.3.1 Methodology

The authors design an internet-based voting system built on the Hyperledger Fabric blockchain framework. Voter authentication employs dual biometric modalities (fingerprint scanning and facial recognition) to prevent impersonation and duplicate voting. The registration process requires voters to submit biometric samples, which are stored as templates for subsequent verification during the voting phase. The blockchain serves as a decentralized and immutable ledger for storing voting records, ensuring transparency and auditability. Votes are encrypted using standard AES-256 encryption before submission to the blockchain. The system architecture follows a client-server model where the biometric verification occurs on the server side before the vote is recorded on the distributed ledger.

2.2.3.2 Contributions

The primary contribution of this work is the practical demonstration of biometric-authenticated blockchain voting with real participants. In testing with 100 participants, the system achieved an 87% successful registration rate using biometrics and approximately 88% of participants successfully cast votes using voter ID paired with either fingerprint or facial recognition. The dual-factor biometric approach provides stronger identity verification than single-factor methods. The use of Hyperledger Fabric ensures enterprise-grade blockchain infrastructure with permissioned access control. The authors also provide a comprehensive analysis of user acceptance and practical deployment challenges.

2.2.3.3 Limitations

Despite its practical contributions, the system's cryptographic foundation is notably weak. The use of basic AES-256 encryption provides confidentiality but does not support homomorphic tallying, meaning votes must be decrypted by a trusted authority for counting. There are no zero-knowledge proofs to verify vote validity without revealing content, no ring signatures for anonymous ballot submission, no coercion resistance mechanism, and no post-quantum security considerations. The 87% registration success rate suggests significant usability challenges with biometric enrollment. Furthermore, the system lacks formal security proofs and relies primarily on the blockchain's immutability for integrity guarantees. The small test scale (100 participants) provides limited evidence for scalability

claims. The privacy model depends entirely on AES encryption and Fabric's channel isolation, offering no mathematical privacy guarantees comparable to homomorphic encryption or differential privacy.

2.2.4 Timed-Release E-Voting with Paillier Homomorphic Encryption

Yuan et al. [22] propose a blockchain-based e-voting scheme that combines Paillier homomorphic encryption with timed-release encryption to control the decryption timeline and eliminate the need for a trusted counting authority.

2.2.4.1 Methodology

The authors integrate two cryptographic primitives: Paillier's additively homomorphic encryption for privacy-preserving vote tallying, and timed-release encryption (TRE) for automated key release. The TRE mechanism controls when the private key becomes available for decryption through an independent microservice, eliminating human intervention in the tallying process. Ballot legitimacy is ensured through partial knowledge proofs that verify each vote is well-formed without revealing its content. The system operates on a blockchain where encrypted votes are submitted as transactions. The homomorphic property of Paillier encryption allows the tally to be computed by multiplying ciphertexts ($E(v_1) \times E(v_2) = E(v_1 + v_2)$), after which the result is decrypted only when the timed-release condition is met. The scheme aims to prevent premature result disclosure that could influence ongoing voting.

2.2.4.2 Contributions

This work makes two primary contributions. First, the integration of Paillier homomorphic encryption with timed-release encryption provides both ballot privacy and temporal fairness: no party can learn intermediate results before the election ends. Second, the partial knowledge proofs ensure that only legitimate ballots are included in the tally without requiring a trusted authority to inspect individual votes. The authors demonstrate semantic security of the scheme and provide feasibility analysis for practical implementation. The use of an automated microservice for key release reduces the trust assumptions compared to systems that rely on election officials to perform decryption.

2.2.4.3 Limitations

While the scheme provides strong privacy through Paillier encryption, it lacks several critical security properties. There are no ring signatures or comparable mechanisms for voter anonymity beyond encryption. The system does not provide coercion resistance: a voter cannot produce a fake credential or plausible alternative vote under duress. The timed-release mechanism introduces a dependency on an external microservice, which, if

compromised, could enable premature decryption. The system provides no post-quantum security, leaving it vulnerable to future quantum attacks on the Paillier cryptosystem. Furthermore, the scheme does not implement threshold decryption, meaning the decryption key must be held by a single entity (or the TRE service), creating a potential single point of failure for ballot secrecy.

2.2.5 Scalable Coercion-Resistant Blockchain Voting

Yin et al. [23] present a scalable coercion-resistant voting scheme specifically designed for blockchain decision-making, published in IEEE Transactions on Dependable and Secure Computing (TDSC). This work addresses the inherent tension between coercion resistance and scalability in blockchain-based voting systems.

2.2.5.1 Methodology

The authors design a voting protocol that achieves coercion resistance by ensuring the randomness used by the voting client cannot serve as a witness or proof of the actual vote to a coercer. The scheme supports private weighted votes and 1-layer liquid democracy, where voters can delegate their voting power to trusted experts. The protocol achieves $O(n)$ overall complexity (where n is the number of voters) and reduces the ballot size to $\Theta(1)$, independent of the number of candidates, a significant improvement over prior work which required $\Theta(m)$ ballot size. The authors provide formal game-based security proofs demonstrating ballot privacy, verifiability, and coercion resistance under standard cryptographic assumptions.

2.2.5.2 Contributions

The main contribution is the first scalable coercion-resistant blockchain voting scheme with $O(n)$ complexity and constant-size ballots. The prototype evaluation demonstrates that the tally procedure is more than $6\times$ faster than previous state-of-the-art schemes such as VoteAgain for elections with over 50,000 voters. The formal security proofs under game-based definitions provide rigorous guarantees. The support for weighted voting and liquid democracy extends the applicability beyond simple majority elections to more complex governance scenarios.

2.2.5.3 Limitations

Despite achieving scalable coercion resistance, the scheme has several limitations. It does not implement post-quantum cryptographic protection, leaving ballot privacy vulnerable to future quantum adversaries. The system lacks homomorphic encryption for privacy-preserving tallying, biometric voter authentication, and anonymous aggregation mechanisms. There is no integration with a production blockchain platform (such as

Hyperledger Fabric) with deployed chaincode and real network benchmarks. The protocol implements only two cryptographic primitives, compared to the seven-protocol composition proposed in our framework. While the game-based formal proofs are stronger than symbolic-model verification, the system does not provide practical deployment validation or comprehensive test coverage.

2.2.6 Survey: Blockchain for Securing Electronic Voting Systems

Ohize, Alfa, and Ibrahim [24] present a comprehensive survey of blockchain-based electronic voting systems, analyzing architectures, trends, solutions, and challenges across the field. Published in Cluster Computing (Springer), this survey provides a systematic overview of the research landscape.

2.2.6.1 Methodology

The authors conduct a systematic literature review of blockchain-based e-voting systems, categorizing existing works along multiple dimensions: blockchain platform (public vs. permissioned), consensus mechanism, cryptographic techniques employed, security properties achieved, and deployment maturity. The survey analyzes both theoretical proposals and implemented systems, identifying common architectural patterns and recurring security trade-offs. The authors evaluate each system against a set of core e-voting requirements including ballot secrecy, voter eligibility verification, universal verifiability, coercion resistance, and receipt-freeness.

2.2.6.2 Contributions

The survey provides a structured taxonomy of blockchain e-voting approaches and identifies several important findings. First, while blockchain provides strong integrity guarantees, most systems still rely on additional cryptographic layers for privacy; blockchain transparency alone is insufficient for ballot secrecy. Second, the survey identifies that coercion resistance and receipt-freeness remain the most challenging properties to achieve in blockchain-based systems, with very few implementations addressing these requirements. Third, scalability remains a persistent concern, with most systems tested only at small scales (hundreds to low thousands of voters). Fourth, the survey notes that post-quantum security is largely ignored in existing implementations, despite being critical for long-term ballot confidentiality.

2.2.6.3 Limitations

As a survey paper, this work does not propose new solutions but rather identifies gaps and challenges. The authors note that the field lacks standardized evaluation frameworks, making direct comparisons between systems difficult. The survey also observes that

most reviewed systems remain at the prototype or proof-of-concept stage, with very few undergoing real-world deployment or large-scale testing. The identified gaps, particularly in coercion resistance, post-quantum security, and scalable cryptographic tallying, directly motivate the design of the proposed framework.

2.3 Background Technology

This section briefly reviews the pre-published cryptographic primitives, protocols, and platforms upon which the proposed framework is constructed. Each subsection identifies the foundational reference, summarises the underlying hardness assumption or mathematical structure, and notes the role the primitive plays in the overall voting pipeline. Detailed protocol-level constructions and parameter choices are deferred to Chapter 3.

2.3.1 Paillier Public-Key Cryptosystem

The Paillier cryptosystem [8] is a probabilistic public-key encryption scheme whose security reduces to the Decisional Composite Residuosity Assumption (DCRA) over the group $\mathbb{Z}_{n^2}^*$, where $n = pq$ is a product of two large primes. Its defining property is *additive homomorphism*: the product of two ciphertexts decrypts to the sum of the underlying plaintexts. This property enables an authority to compute encrypted vote totals without ever observing individual ballots. The scheme has been adopted by a number of e-voting designs because it permits exact tallying, in contrast to approximate techniques such as differential privacy.

2.3.2 Pedersen Commitments

A Pedersen commitment [17] binds a committer to a secret value m via $C = g^m h^r \bmod p$, where g and h are generators of a prime-order subgroup and r is fresh randomness. The construction is *perfectly binding* (no two openings exist for a given C) and *computationally hiding* under the discrete logarithm assumption. Pedersen commitments serve as the cryptographic substrate for the zero-knowledge proofs used to verify ballot well-formedness and for the deniable credentials that underpin coercion resistance in this work.

2.3.3 Zero-Knowledge Σ-Protocols

A Σ-protocol is a three-move public-coin proof system (commit-challenge-response) for languages defined by a discrete-logarithm relation. The Schnorr proof of knowledge [25] is the canonical instance and provides the building block for the binary and sum proofs employed here. The Fiat-Shamir transformation [26] renders such proofs non-interactive by replacing the verifier's challenge with the output of a cryptographic hash function. Bernhard, Pereira, and Warinschi [9] demonstrated that naïve applications of Fiat-Shamir in e-voting systems are vulnerable to malleability attacks; their *Strong Fiat-Shamir* variant,

which incorporates the public parameters and statement into the hash input, is therefore used throughout the proposed framework.

2.3.4 Linkable Ring Signatures

Ring signatures, introduced by Rivest, Shamir, and Tauman [27], allow a signer to produce a signature on behalf of an ad-hoc group of public keys without revealing which group member produced it. *Linkable* ring signatures, formalised by Liu, Wei, and Wong [10], additionally publish a deterministic key image $I = H(\mathrm{pk})^{\mathrm{sk}}$ that allows two signatures by the same signer to be detected without breaking anonymity. This dual property, anonymity within the ring together with verifiable non-duplication, is precisely what e-voting requires for ballot secrecy and double-vote prevention.

2.3.5 Threshold Cryptography and Secret Sharing

Shamir's (t, n)-secret sharing scheme [28] distributes a secret among n parties such that any t shares reconstruct it via Lagrange interpolation while any $t - 1$ shares reveal nothing. Damgård, Jurik, and Nielsen [18] extended Paillier's scheme to a threshold variant in which the decryption key is split across talliers, each contributing a partial decryption together with a non-interactive zero-knowledge proof of correctness. The proposed framework adopts this DJN construction so that no single trustee can unilaterally decrypt election results.

2.3.6 Cryptographic Hash Functions and Message Authentication

The security of the Fiat-Shamir transform, the ring signature key image, and the SMDC index derivation rely on cryptographic hash functions. SHA-256, standardised in NIST FIPS 180-4 [29], and the Keccak-based SHA-3 family in FIPS 202 [30] are used as random oracles in this work. For keyed message authentication, HMAC [31, 32] provides existential unforgeability under the assumption that the underlying compression function is a pseudorandom function. HMAC-SHA256 is used both for SMDC slot index derivation and for the constant-time duress score computation introduced in Chapter 3.

2.3.7 Post-Quantum Key Encapsulation: ML-KEM (Kyber)

Shor's polynomial-time quantum algorithms for integer factorisation and discrete logarithms [13] undermine the long-term confidentiality of ciphertexts produced by Paillier, RSA, and elliptic-curve schemes. ML-KEM, standardised by NIST as FIPS 203 [12] and based on the CRYSTALS-Kyber proposal, is a key-encapsulation mechanism whose security reduces to the Module Learning-With-Errors (Module-LWE) problem over algebraic lattices. The Kyber-768 parameter set targets NIST security category 3 (equivalent to

AES-192) and is the variant adopted in this work to provide a quantum-resistant binding for vote ciphertexts in transit and at rest.

2.3.8 Authenticated Symmetric Encryption: AES-GCM

For non-homomorphic payloads (audit metadata, transport-level confidentiality), the proposed framework uses AES in Galois/Counter Mode (AES-GCM) as specified in NIST SP 800-38D [33]. AES-GCM provides IND-CCA2 confidentiality and INT-CTXT integrity in a single pass, with hardware-accelerated performance on commodity processors.

2.3.9 Hyperledger Fabric

Hyperledger Fabric [1] is a permissioned distributed ledger platform with a modular architecture that separates transaction execution, ordering, and validation. Smart contracts (*chaincode*) execute in isolated containers; ordering is performed by a pluggable consensus service (Raft in the present deployment); and validation is performed independently by each peer. Recent empirical studies [7] have measured Fabric's throughput on the order of hundreds to thousands of transactions per second, which is several orders of magnitude beyond the throughput attainable on public proof-of-work blockchains. The combination of permissioned membership, configurable endorsement, and high throughput makes Fabric the natural deployment target for an election-grade voting system.

2.3.10 Samplable Anonymous Aggregation

Samplable Anonymous Aggregation (SA^2) [11] is a two-server cryptographic primitive originally proposed by Apple for privacy-preserving federated data analysis, building on the secure aggregation construction of Bonawitz et al. [34]. Each client splits its contribution into two additive shares blinded by a fresh mask; one share is delivered to a *leader* server and the complementary share to a *helper* server. Provided the two servers do not collude, neither can reconstruct any individual contribution while their combined output yields the exact aggregate. The proposed framework adapts this primitive from federated learning to encrypted voting, a novel use described in Chapter 3.

2.4 Comparison Among State-of-the-Art Works

Table 2.1 presents a comparative analysis of the reviewed e-voting systems across key security properties and implementation characteristics. This comparison highlights the gaps in existing solutions that The proposed framework addresses through its seven-protocol cryptographic stack.

Table 2.1: Comparison of State-of-the-Art Blockchain E-Voting Approaches

Feature	**ISE** [19]	**BP-Vot** [20]	**Faruk** [21]	**Yuan** [22]	**Yin** [23]	**Ours**
Homomorphic Enc.	✗	✗	✗	✓	✗	✓
Zero-Knowledge	✗	✗	✗	✓†	✗	✓
Ring Signatures	✓	✗	✗	✗	✗	✓
Coercion Resist.	✗	✗	✗	✗	✓	✓
Post-Quantum	✓†	✗	✗	✗	✗	✓
Biometric Auth.	✗	✗	✓	✗	✗	✓†
Anonymous Agg.	✗	✗	✗	✗	✗	✓
Threshold Dec.	✗	✗	✗	✗	✗	✓
Formal Proofs	✗	✗	✗	✗	✓	✓
Exact Tally	✓	✗	✓	✓	✓	✓
Blockchain Deploy	✗	✓	✓	✗	✗	✓
Complexity	$O(nm^2)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Protocol Count	2	2	1	2	2	**7**
Implementation	✓	✓	✓	✓†	✓†	✓

✓ = full support, ✗ = no support, ✓† = partial support, $O(\cdot)$ = tally complexity.

The "Ours" Biometric Authentication entry is marked partial (✓†) rather than full (✓). The proposed framework includes a biometric pipeline (`internal/biometric`) that performs SHA3 hash-equality comparison of fingerprint byte input against a stored per-voter hash, with constant-time verification to prevent timing side channels and a liveness pre-check. However, this construction is a hash-equality stub rather than a full biometric matcher: it does not perform feature extraction, fuzzy minutiae matching, or a distance-metric comparison, and the False Acceptance Rate (FAR), False Rejection Rate (FRR), and enrollment success rate have not been empirically characterised through a user trial. Faruk et al. [21] provide a 100-participant trial reporting an 87% biometric registration success rate, which the present work does not match. The hash-equality stub is suitable for the cryptographic-pipeline integration scope of this thesis but should not be interpreted as a deployment-ready biometric matcher; characterising FAR, FRR, and enrollment success rates with a user study is identified as future work in Chapter 5.

From Table 2.1, several observations can be made. First, no existing system provides all the security properties simultaneously. ISE-Voting offers ring signatures but lacks homomorphic encryption and coercion resistance. BP-Vot provides differential privacy but sacrifices exact tallying. Faruk et al. offer biometric authentication with a 100-participant user trial but with minimal cryptographic depth. Yuan et al. implement Paillier homomorphic encryption but without anonymity or coercion resistance. Yin et al. [23] represent the closest competitor to our framework, achieving both $O(n)$ complexity and coercion resistance with formal game-based proofs; however, their system lacks homomorphic tallying, post-quantum protection, biometric authentication, anonymous aggregation, and production blockchain deployment.

the proposed framework is the only system that integrates all reviewed security properties (homomorphic encryption, zero-knowledge proofs, ring signatures, coercion resistance, post-quantum protection, biometric authentication, anonymous aggregation, and threshold decryption) into a single, functional voting pipeline with empirical blockchain performance benchmarks. Notably, The proposed framework integrates **seven** cryptographic protocols, compared to at most two in any existing system, and provides a full implementation with production blockchain deployment and ProVerif formal verification.

2.5 Summary

In this chapter, we reviewed six key works representing the current state of the art in blockchain-based e-voting. We analyzed each work's methodology, contributions, and limitations with a focus on security properties, cryptographic techniques, and practical deployment.

The review reveals that while notable progress has been made in individual security dimensions (privacy through homomorphic encryption, anonymity through ring signatures, and verifiability through zero-knowledge proofs), no existing system offers a complete security framework that simultaneously addresses all critical requirements. In particular, combining coercion resistance with exact homomorphic tallying and post-quantum security remains an open challenge.

Specifically, the literature analysis identifies the following critical gaps that The proposed framework addresses:

1. **Coercion Resistance Gap**: While Yin et al. [23] achieve coercion resistance with formal proofs, their system lacks homomorphic tallying, post-quantum protection, and real blockchain deployment. Our framework fills this gap through the SMDC deniable credential mechanism integrated with Paillier homomorphic tallying and Kyber768 post-quantum encryption on a production Hyperledger Fabric network.

2. **Post-Quantum Gap**: No existing blockchain e-voting system provides post-quantum security with a standardized algorithm. The proposed framework integrates Kyber768 (ML-KEM, NIST FIPS 203) hybrid encryption.

3. **Protocol Integration Gap**: Existing systems implement at most two cryptographic protocols. The proposed framework integrates seven protocols while maintaining $O(n)$ tally complexity and sub-100ms per-vote pipeline time.

4. **Empirical Validation Gap**: Most systems provide only theoretical analysis or small-scale simulations. The proposed framework provides comprehensive empirical benchmarks on production Hyperledger Fabric infrastructure with up to 100,000 transactions.

The comparison table (Table 2.1) quantifies these gaps and demonstrates that The proposed framework addresses all identified limitations through its seven-protocol cryptographic stack. The next chapter presents the detailed system design and mathematical formulation of our framework, including the novel SMDC coercion resistance mechanism and SA^2 anonymous aggregation protocol.

3. System Design and Cryptographic Protocols

3.1 Introduction

This chapter presents the complete system design of the proposed framework, a blockchain-based e-voting system, deployed on Hyperledger Fabric [1], that integrates seven cryptographic protocols into a unified voting pipeline. Section 3.2 describes the overall system framework and assumptions. Section 3.3 defines the mathematical notation used throughout this chapter. Section 3.4 presents the detailed formulation of each cryptographic protocol. Section 3.5 describes the 17-step vote casting pipeline. Section 3.6 details the Hyperledger Fabric blockchain integration. Section 3.7 addresses engineering and societal considerations. Section 3.8 presents the formal security analysis with six theorems and proof sketches. Section 3.9 concludes the chapter.

3.2 Proposed Framework and Assumptions

The proposed framework consists of four architectural layers that interact to provide comprehensive voting security, as illustrated in Figure 3.1. The layers are: (1) the Authentication Layer, which handles voter identity verification through biometric and credential-based mechanisms; (2) the Cryptographic Computation Layer, which executes the seven-protocol pipeline for each vote; (3) the Privacy Layer, which implements SA^2 anonymous aggregation and SMDC deniable credentials; and (4) the Blockchain Storage Layer, which records encrypted vote records on Hyperledger Fabric v2.5.

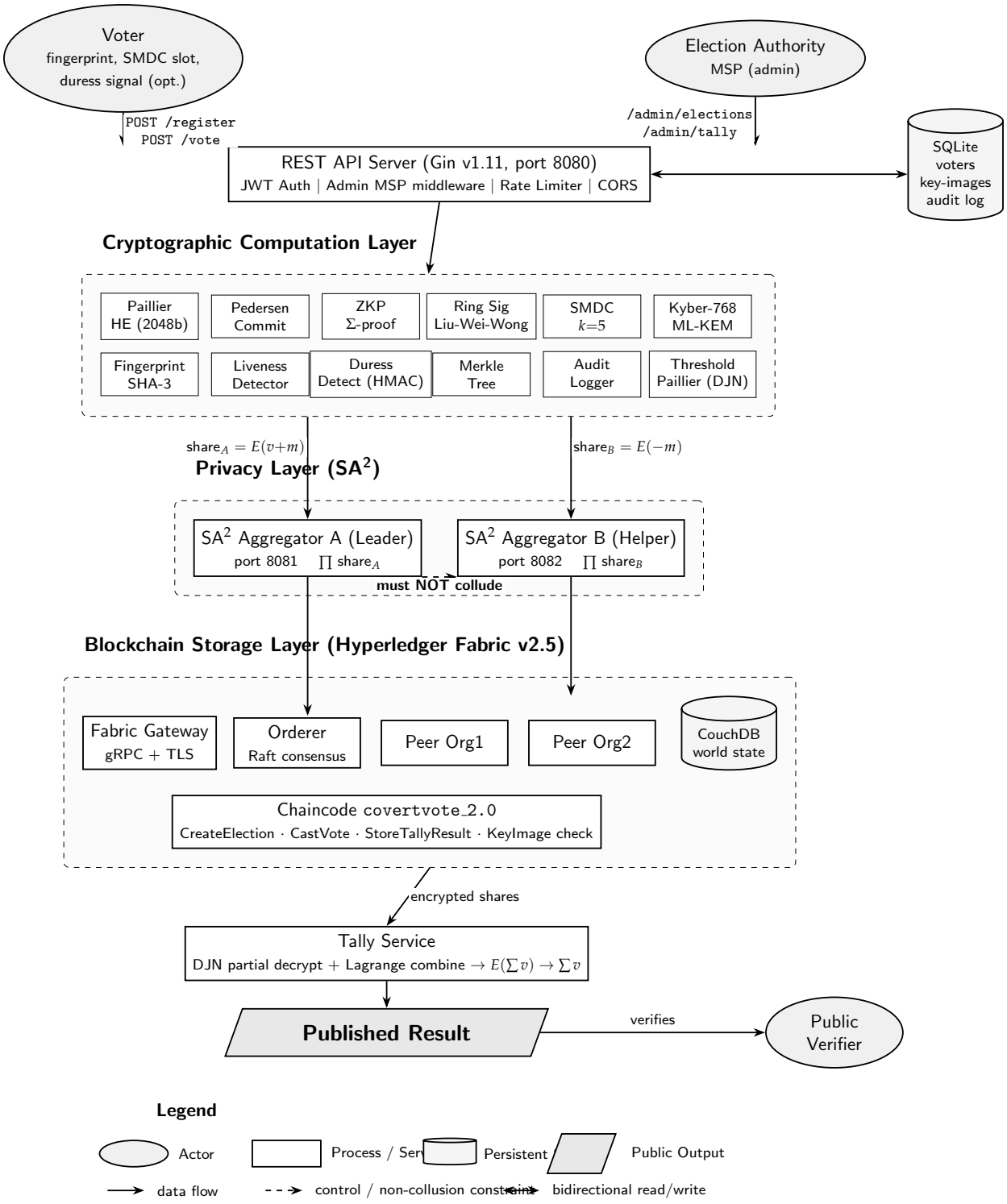

Fig. 3.1: End-to-end system architecture of the proposed framework. The diagram traces the full path from voter authentication through the cryptographic pipeline, the two non-colluding SA^2 aggregators, the Hyperledger Fabric storage layer, and the threshold tally service to the published result. All shapes and line styles are distinguishable in greyscale.

The framework operates under the following assumptions:

1. The blockchain network consists of at least one ordering node and two peer nodes running Hyperledger Fabric v2.5 [1] with Raft consensus [35].

2. The two SA^2 aggregation servers are operated by independent, non-colluding entities. The security guarantee holds as long as at least one server remains honest.

3. Voters have access to a secure device for biometric authentication and vote submission.

4. The election authority generates and distributes SMDC credentials to eligible voters before the election begins.

5. The Paillier key pair is generated with a minimum of 2048-bit modulus for computational security.

6. Each voter is assigned $k = 5$ credential slots (1 real, 4 fake) for coercion resistance.

3.3 Notation

Table 3.1 summarizes the mathematical symbols used throughout this chapter.

Table 3.1: Mathematical Notation

Symbol	**Meaning**
p, q	Large primes for key generation
n	RSA modulus $n = p \times q$ (Paillier)
g	Generator; $g = n + 1$ for Paillier, subgroup generator for Pedersen
h	Independent generator for Pedersen ($\log_g h$ unknown)
λ	$\text{lcm}(p-1, q-1)$ (Paillier private key component)
μ	$L(g^\lambda \bmod n^2)^{-1} \bmod n$ (Paillier decryption parameter)
$L(x)$	$L(x) = (x-1)/n$ (Paillier L-function)
$E(m)$	Paillier encryption of message m: $g^m \cdot r^n \bmod n^2$
C	Pedersen commitment: $C = g^m \cdot h^r \bmod p$
r	Randomness for encryption or commitment
w_i	Vote weight for candidate i; $w_i \in \{0, 1\}$, $\sum w_i = 1$
k	Number of SMDC credential slots (default $k = 5$)
I	Key image for linkable ring signature: $I = H(\text{pk})^{\text{sk}}$
σ	Ring signature tuple $(I, c_0, r_0, r_1, \ldots, r_{n-1})$
pk_i, sk_i	Public/private key pair for ring member i
$\mathcal{R}$	Ring of public keys $\{\text{pk}_1, \ldots, \text{pk}_n\}$ with $\lvert\mathcal{R}\rvert = 100$
S_A, S_B	SA^2 vote shares for Server A and Server B
mask	Random masking value for SA^2 share splitting

3.4 Cryptographic Protocol Formulation

This section presents the mathematical formulation of all seven cryptographic protocols implemented in our framework. Each protocol provides a specific security guarantee, and their composition forms the complete voting pipeline.

3.4.1 Paillier Homomorphic Encryption

The Paillier cryptosystem [8] is an additively homomorphic public-key encryption scheme based on the Decisional Composite Residuosity Assumption (DCRA). It enables the computation of vote tallies directly on encrypted ballots without decryption.

3.4.1.1 Key Generation

The key generation process proceeds as in the original construction of Paillier [8]:

1. Select two large primes p and q of equal length (≥ 1024 bits each), ensuring $p \neq q$.
2. Compute the RSA modulus:
$$n = p \times q \tag{3.1}$$
3. Compute Carmichael's function:
$$\lambda = \text{lcm}(p-1, q-1) \tag{3.2}$$
4. Set the generator $g = n + 1$ (standard choice that simplifies computation).
5. Compute the decryption parameter:
$$\mu = L(g^{\lambda} \bmod n^2)^{-1} \bmod n \tag{3.3}$$
where $L(x) = \frac{x-1}{n}$.

The public key is (n, g, n^2) and the private key is (λ, μ, p, q).

3.4.1.2 Encryption

Encryption of a plaintext message $m \in \mathbb{Z}_n$ proceeds as defined by Paillier [8]:

1. Select a random $r \in \mathbb{Z}_n^*$ where $r > 0$.
2. Compute the ciphertext:
$$c = g^m \cdot r^n \bmod n^2 \tag{3.4}$$

3.4.1.3 Decryption

Decryption follows the inverse construction of Paillier [8]:

$$m = L(c^{\lambda} \bmod n^2) \cdot \mu \bmod n \tag{3.5}$$

3.4.1.4 Homomorphic Properties

The Paillier cryptosystem supports the following homomorphic operations [8]:

Additive Homomorphism. The product of two ciphertexts decrypts to the sum of their plaintexts:

$$E(m_1) \cdot E(m_2) \bmod n^2 = E(m_1 + m_2) \tag{3.6}$$

This property is the foundation of the proposed framework's $O(n)$-per-candidate (and $O(n \cdot m)$ total over m candidates) tally. Given n encrypted votes $\{E(v_1), E(v_2), \ldots, E(v_n)\}$ for a single candidate, the encrypted tally is:

$$E\left(\sum_{i=1}^{n} v_i\right) = \prod_{i=1}^{n} E(v_i) \bmod n^2 \tag{3.7}$$

Scalar Multiplication. A ciphertext raised to a scalar k yields the encryption of $k \times m$:

$$E(m)^k \bmod n^2 = E(k \cdot m) \tag{3.8}$$

3.4.2 Pedersen Commitments

Pedersen commitments [17] provide computationally hiding and perfectly binding commitments, serving as the foundation for zero-knowledge proofs in our framework.

3.4.2.1 Parameter Generation

1. Generate a safe prime $p = 2q + 1$ where q is also prime.
2. Find a generator g of the subgroup of order q: select random $h \in \mathbb{Z}_p^*$ and compute $g = h^{(p-1)/q} \bmod p$ until $g \neq 1$.
3. Derive an independent generator h such that $\log_g h$ is unknown, using hash-to-group: $h = \text{SHA3}(g)^{(p-1)/q} \bmod p$.

3.4.2.2 Commitment

A commitment to a value $m \in \mathbb{Z}_q$ with randomness $r \in \mathbb{Z}_q$ takes the form defined by Pedersen [17]:

$$C = g^m \cdot h^r \bmod p \tag{3.9}$$

Security Properties:

- **Computationally Hiding**: Given C, no polynomial-time adversary can determine m (under the Discrete Logarithm assumption).
- **Perfectly Binding**: There exists no $(m', r') \neq (m, r)$ such that $g^{m'} \cdot h^{r'} = g^m \cdot h^r$ (information-theoretically).

Homomorphic Property [17]**:**

$$C_1 \cdot C_2 = g^{m_1+m_2} \cdot h^{r_1+r_2} \bmod p = \text{Commit}(m_1 + m_2, r_1 + r_2) \tag{3.10}$$

3.4.3 Zero-Knowledge Proofs (Σ-Protocol)

Our framework implements two zero-knowledge proofs built from the Schnorr proof of knowledge of a discrete logarithm [25] and rendered non-interactive by the Fiat-Shamir transformation [26]. Following the recommendation of Bernhard, Pereira, and Warinschi [9], the *Strong* variant of Fiat-Shamir is used, which incorporates the public parameters and statement into the hash input to prevent malleability. Two proofs are required per ballot: a binary proof that each per-candidate weight $w_i \in \{0, 1\}$, and a sum proof that $\sum_i w_i = 1$.

3.4.3.1 Binary Proof

The binary proof demonstrates that a Pedersen commitment $C = g^w \cdot h^r$ contains either $w = 0$ or $w = 1$, without revealing which. It is constructed as an OR-composition of two Schnorr proofs [25] and made non-interactive via the Fiat-Shamir transform [26].

Proof Generation (for $w = 0$; the $w = 1$ case is symmetric):

1. **Simulate** the $w = 1$ branch: choose random $d_1, r_1 \in \mathbb{Z}_q$ and compute:

$$A_1 = g \cdot h^{r_1} \cdot (C/g)^{-d_1} \bmod p \tag{3.11}$$

2. **Real proof** for $w = 0$: choose random $r_0 \in \mathbb{Z}_q$ and compute:

$$A_0 = h^{r_0} \bmod p \tag{3.12}$$

3. Compute the Strong Fiat-Shamir challenge [26, 9] using a SHA-256 random oracle [29]:

$$c = H(\text{params} \| \text{nonce} \| \text{electionID} \| C \| A_0 \| A_1) \bmod q \tag{3.13}$$

   where params $= (p, q, g, h)$ are included per the Bernhard-Pereira-Warinschi recommendation [9].

4. Compute $d_0 = c - d_1 \bmod q$ and $f_0 = r_0 + d_0 \cdot r \bmod q$.

The proof is $\pi = (A_0, A_1, d_0, d_1, f_0, f_1, \text{nonce}, \text{electionID})$.

Verification:

1. Check $d_0 + d_1 \equiv c \pmod{q}$
2. Check $h^{f_0} \cdot C^{-d_0} \equiv A_0 \pmod{p}$
3. Check $g \cdot h^{f_1} \cdot (C/g)^{-d_1} \equiv A_1 \pmod{p}$

3.4.3.2 Sum Proof

The sum proof demonstrates that $\sum_{i=1}^{m} w_i = 1$ across all candidate commitments, ensuring each voter selects exactly one candidate.

Given commitments $\{C_1, \ldots, C_m\}$ with randomness $\{r_1, \ldots, r_m\}$:

1. Compute the product: $\Pi = \prod_{i=1}^{m} C_i = g^{\sum w_i} \cdot h^{\sum r_i} \bmod p$
2. Since $\sum w_i = 1$, we have $\Pi = g \cdot h^R \bmod p$ where $R = \sum r_i \bmod q$.
3. Execute a Schnorr proof of knowledge [25] of R: choose random k, compute $A = h^k$, challenge $c = H(\text{params} \| \Pi \| A \| g)$ via Fiat-Shamir [26], response $s = k + c \cdot R \bmod q$.

Verification: Recompute A from (s, c): check $h^s \cdot (\Pi/g)^{-c} \equiv A \pmod{p}$.

3.4.4 Linkable Ring Signatures

The framework employs linkable ring signatures, a refinement [10] of the original spontaneous-anonymous-group signature of Rivest, Shamir, and Tauman [27]. A fixed ring size of $|\mathcal{R}| = 100$ members is used. The hash function $H(\cdot)$ in equations (3.15) and the signing procedure is instantiated with SHA-256 [29].

3.4.4.1 Key Generation

Each voter generates a key pair following the Liu-Wei-Wong construction [10]:

$$\text{sk} \xleftarrow{\$} \mathbb{Z}_q, \quad \text{pk} = g^{\text{sk}} \bmod p \tag{3.14}$$

3.4.4.2 Signing

Given message M, signer key $(\text{sk}_s, \text{pk}_s)$, ring $\mathcal{R} = \{\text{pk}_0, \ldots, \text{pk}_{n-1}\}$, and signer index s, the signing procedure follows [10]:

1. Compute the key image (for double-vote detection):

$$I = H(\text{pk}_s)^{\text{sk}_s} \bmod p \tag{3.15}$$

2. Choose random $\alpha \in \mathbb{Z}_q$ and compute:

$$L_s = g^{\alpha} \bmod p, \quad R_s = H(\text{pk}_s)^{\alpha} \bmod p \tag{3.16}$$

3. Set $c_{s+1} = H(M \| L_s \| R_s)$.

4. For each non-signer $i \neq s$ (in cyclic order), choose random r_i and compute:

$$L_i = g^{r_i} \cdot \mathrm{pk}_i^{c_i} \bmod p, \quad R_i = H(\mathrm{pk}_i)^{r_i} \cdot I^{c_i} \bmod p \tag{3.17}$$

$$c_{i+1} = H(M \| L_i \| R_i) \tag{3.18}$$

5. Close the ring: $r_s = \alpha - c_s \cdot \mathrm{sk}_s \bmod q$.

The signature is $\sigma = (I, c_0, r_0, r_1, \ldots, r_{n-1})$.

3.4.4.3 Verification and Linkability

Verification reconstructs all (L_i, R_i) pairs and checks that the ring closes: the final computed challenge equals c_0. Linkability is achieved through key images: two signatures from the same signer produce identical I values, enabling double-vote detection via $I_1 \stackrel{?}{=} I_2$.

3.4.5 SMDC: Self-Masking Deniable Credentials

The construction presented in this subsection is a novel contribution of this thesis. While the underlying cryptographic primitives (Pedersen commitments, Σ-proofs, and HMAC) are standard, their composition into a deniable-credential mechanism with k-slot self-masking, deterministic HMAC-derived real-slot indexing, and silent-discard semantics for coerced votes is, to the best of our knowledge, original to this work.

Each voter receives $k = 5$ credential slots, of which exactly one is real (weight $= 1$) and the remaining four are fake (weight $= 0$). The real slot index is derived deterministically via HMAC-SHA256 [31, 32, 29] and is never stored, ensuring only the legitimate voter can re-derive it from $(\mathrm{vid}, \mathrm{eid})$.

3.4.5.1 Credential Generation

For voter with identifier vid in election eid:

1. **Derive real index** (deterministic, re-derivable):

$$\mathrm{idx}_{\mathrm{real}} = \text{HMAC-SHA256}(\mathrm{eid}, \text{“smdc-real-index:”} \| \mathrm{vid} \| \mathrm{eid}) \bmod k \tag{3.19}$$

2. **Create k slots**: For each slot $i \in \{0, \ldots, k-1\}$:

$$w_i = \begin{cases} 1 & \text{if } i = \mathrm{idx}_{\mathrm{real}} \\ 0 & \text{otherwise} \end{cases} \tag{3.20}$$

3. **Commit**: Generate a Pedersen commitment [17] for each slot:

$$C_i = g^{w_i} \cdot h^{r_i} \bmod p, \quad r_i \xleftarrow{\$} \mathbb{Z}_q \tag{3.21}$$

4. **Prove binary**: For each C_i, generate a ZK proof π_i^{bin} that $w_i \in \{0, 1\}$.

5. **Prove sum**: Generate proof π^{sum} that $\sum_{i=0}^{k-1} w_i = 1$.

The credential is $\text{Cred} = (\text{vid}, k, \{(C_i, \pi_i^{\text{bin}})\}_{i=0}^{k-1}, \pi^{\text{sum}})$.

3.4.5.2 Coercion Resistance Property

Under coercion, the voter presents a *fake* slot $j \neq \text{idx}_{\text{real}}$. The coercer observes:

- A valid Pedersen commitment $C_j = g^0 \cdot h^{r_j} = h^{r_j}$
- A valid binary proof π_j^{bin} (proving $w_j \in \{0, 1\}$, without revealing $w_j = 0$)
- A valid sum proof π^{sum} (proving exactly one slot has $w = 1$)

Due to Pedersen's **computational hiding** property, the coercer cannot determine whether C_j commits to 0 or 1. The vote cast with a fake credential is silently discarded during tally (it contributes $w = 0$ to the homomorphic sum), while the voter's receipt appears identical to a real vote receipt. Figure 3.2 illustrates this mechanism.

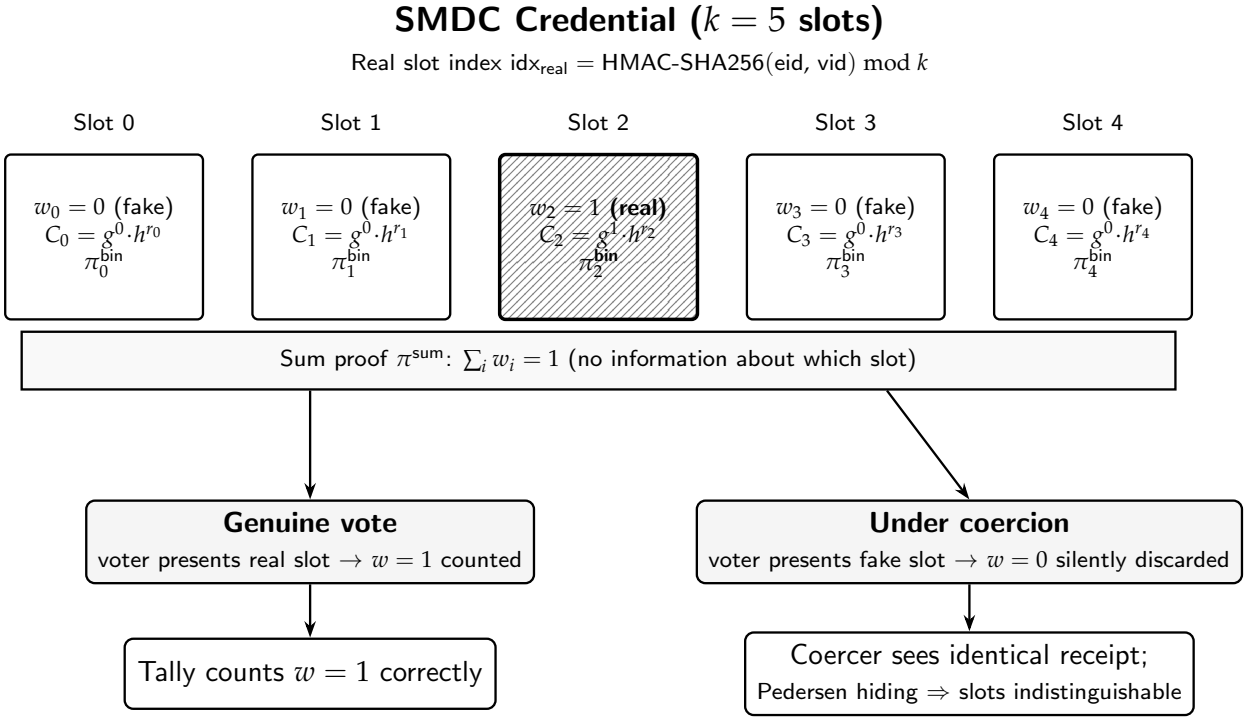

Fig. 3.2: SMDC deniable credential structure with $k = 5$ slots. The real slot is distinguishable only via the cross-hatched fill in this diagram; cryptographically, the Pedersen hiding property renders the real and fake slots computationally indistinguishable to a coercer.

3.4.5.3 Credential Verification

The election system verifies credentials without learning the real index:

1. Verify all k binary proofs: $\forall i : \text{VerifyBinary}(C_i, \pi_i^{\text{bin}}) = \text{true}$
2. Verify the sum proof: $\text{VerifySumOne}(\{C_i\}, \pi^{\text{sum}}) = \text{true}$

3.4.6 SA^2: Samplable Anonymous Aggregation

This subsection describes the integration of the existing SA^2 primitive of Apple Machine Learning Research [11]—which itself builds on the secure aggregation construction of Bonawitz et al. [34]—into the proposed voting pipeline. *The two-server, non-colluding mask-cancellation construction of SA^2 is not a contribution of this thesis; it is invoked as an existing primitive.* The integration choice made here is to pair Paillier additive ciphertexts $E(v)$ with the SA^2 mask-cancellation across the two aggregators in equations (3.22)–(3.24), so that the encrypted tally is reconstructed via mask homomorphism rather than via plaintext secret sharing (the original federated-learning setting).

3.4.6.1 Vote Splitting

Given an encrypted vote $E(v)$ for voter u, the proposed mask-cancellation construction (adapted from Apple's SA^2 [11] which itself builds on the secure-aggregation protocol of Bonawitz et al. [34]) operates as follows:

1. Generate a random mask: $\text{mask} \xleftarrow{\$} \mathbb{Z}_{n/100}$
2. Compute Share A (for Server A):

$$S_A = E(v) \cdot E(\text{mask}) = E(v + \text{mask}) \bmod n^2 \tag{3.22}$$

3. Compute Share B (for Server B):

$$S_B = E(-\text{mask}) = E(n - \text{mask}) \bmod n^2 \tag{3.23}$$

3.4.6.2 Reconstruction

Due to Paillier's additive homomorphism [8]:

$$S_A \cdot S_B = E(v + \text{mask}) \cdot E(-\text{mask}) = E(v) \bmod n^2 \tag{3.24}$$

Security Guarantee: As long as at least one of the two servers remains honest and does not share its received share with the other server, no individual vote v can be reconstructed. Server A sees only $E(v + \text{mask})$ and Server B sees only $E(-\text{mask})$; neither can derive $E(v)$ alone.

3.4.7 Kyber768 Post-Quantum Hybrid Encryption

To protect against the *harvest-now-decrypt-later* threat in which an adversary stores ciphertexts today and decrypts them once a sufficiently large quantum computer becomes available, the framework employs ML-KEM-768 (CRYSTALS-Kyber, NIST FIPS 203 [12]) to derive a post-quantum integrity key for each vote.

3.4.7.1 Hardness Assumption and Parameter Selection

ML-KEM is a key encapsulation mechanism whose IND-CCA2 security reduces to the Module Learning-With-Errors (Module-LWE) and Module Short-Integer-Solution (Module-SIS) problems over polynomial rings $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$ and $q = 3329$. No polynomial-time quantum algorithm is currently known for either problem, in contrast to the integer factorisation and discrete logarithm assumptions broken by Shor's algorithm [13].

NIST defines five security categories. Category 1 matches AES-128 [4], Category 3 matches AES-192, and Category 5 matches AES-256. Kyber-768 is the Category 3 parameter set with module rank $k = 3$, ciphertext size 1088 bytes, and public-key size 1184 bytes. Category 3 is selected for the proposed framework as a balance between long-term security margin and on-the-wire ciphertext size: Category 1 is regarded as conservative for ballots that may need to remain confidential beyond 2050, whereas Category 5 imposes a $\sim$30% larger ciphertext without a corresponding gain in voting-specific assurance.

3.4.7.2 KEM-DEM Hybrid Construction

The hybrid follows the standard KEM-DEM paradigm: the post-quantum KEM transports a fresh symmetric key, which is then used in a classical authenticator over the Paillier ciphertext. The procedure operates as follows:

1. **Kyber Key Generation**: Generate a Kyber768 key pair $(\mathrm{pk}_{\mathrm{ky}}, \mathrm{sk}_{\mathrm{ky}})$ providing NIST Level 3 security (equivalent to AES-192).

2. **Paillier Encryption** [8]: Encrypt the vote with Paillier to preserve homomorphic properties:
$$E(v) = g^v \cdot r^n \bmod n^2 \tag{3.25}$$

3. **Encapsulation** [12]: Generate a shared secret K and ciphertext $\mathrm{ct}_{\mathrm{ky}}$:
$$(K, \mathrm{ct}_{\mathrm{ky}}) = \mathrm{Kyber.Encapsulate}(\mathrm{pk}_{\mathrm{ky}}) \tag{3.26}$$

4. **Key Derivation**: Derive an authentication key $K_{\mathrm{auth}} = \mathrm{SHA256}(K \| \mathrm{salt})$ [29].

5. **MAC Authentication** [31, 32]: Compute HMAC-SHA256 over the Paillier ciphertext:
$$\mathrm{MAC} = \mathrm{HMAC\text{-}SHA256}(K_{\mathrm{auth}}, E(v)) \tag{3.27}$$

6. **Transmission**: Send $(\text{ct}_{\text{ky}}, E(v), \text{salt}, \text{MAC})$.

This hybrid construction ensures dual-layer protection: the Paillier ciphertext $E(v)$ provides semantic security under DCRA and preserves the homomorphic structure required for tallying, while the Kyber-768 layer provides a post-quantum integrity binding via the HMAC. An adversary must compromise *both* layers to tamper with votes undetected; an attacker who later acquires the Kyber secret key (or solves Module-LWE on a future quantum computer) is still confronted with the unmodified Paillier ciphertext, which carries no plaintext information beyond what the adversary already obtains from the public tally. The generic message encryption functions additionally use AES-256-GCM [33] for authenticated symmetric encryption of non-homomorphic payloads such as audit metadata.

3.4.7.3 Scope and Limitations of Post-Quantum Coverage

The ML-KEM-768 layer described above provides post-quantum integrity for vote ciphertexts in transit and at rest. However, the homomorphic tallying remains classically secure under DCRA, and the linkable ring signatures rely on the discrete logarithm assumption, both of which are vulnerable to Shor's algorithm. A future-proof migration would replace these components with lattice-based alternatives: a fully homomorphic encryption scheme (e.g., BGV or CKKS) for tallying, and ML-DSA [14] or a lattice-based linkable ring signature scheme such as those built from Module-SIS. This migration is identified as future work in Chapter 5.

3.4.8 Threshold Paillier Decryption

To prevent any single entity from decrypting election results, Our framework implements the Damgård-Jurik-Nielsen (DJN) [18] threshold decryption scheme with Shamir's Secret Sharing.

3.4.8.1 Key Distribution

The private key λ is split into n shares using (t, n)-Shamir Secret Sharing [28], where at least t parties must cooperate for decryption. Each party i receives share λ_i.

3.4.8.2 Partial Decryption

Each party i computes a partial decryption with a ZK proof of correctness:

$$d_i = c^{\lambda_i} \bmod n^2 \tag{3.28}$$

3.4.8.3 Combination

Given t valid partial decryptions, Lagrange interpolation [28, 18] reconstructs the plaintext:

$$m = L\left(\prod_{i \in S} d_i^{\delta_i} \bmod n^2\right) \cdot \mu \bmod n \tag{3.29}$$

where $\delta_i = \prod_{j \in S, j \neq i} \frac{j}{j-i}$ are the Lagrange coefficients defined for the share evaluation points [28] and S is the set of participating parties.

3.5 Vote Casting Pipeline

Figure 3.3 illustrates the four-phase pipeline, and Algorithm 1 presents the formal pseudocode.

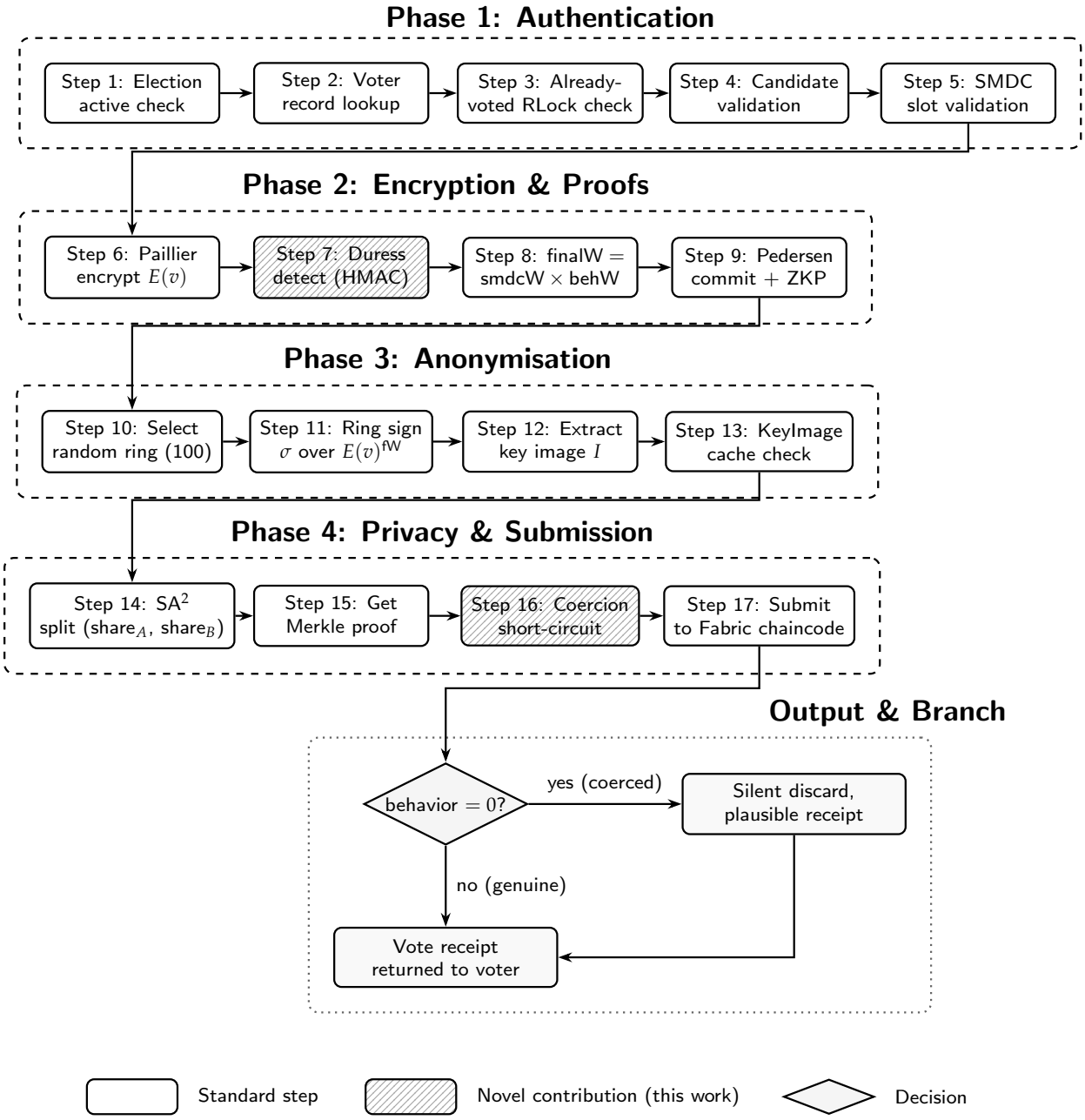

Fig. 3.3: The proposed 17-step vote-casting pipeline, organised into four phases. Hatched boxes denote novel contributions of this thesis (duress detection, coercion short-circuit). The diamond marks the duress decision branch leading to either persistent submission or silent discard with a plausible receipt.

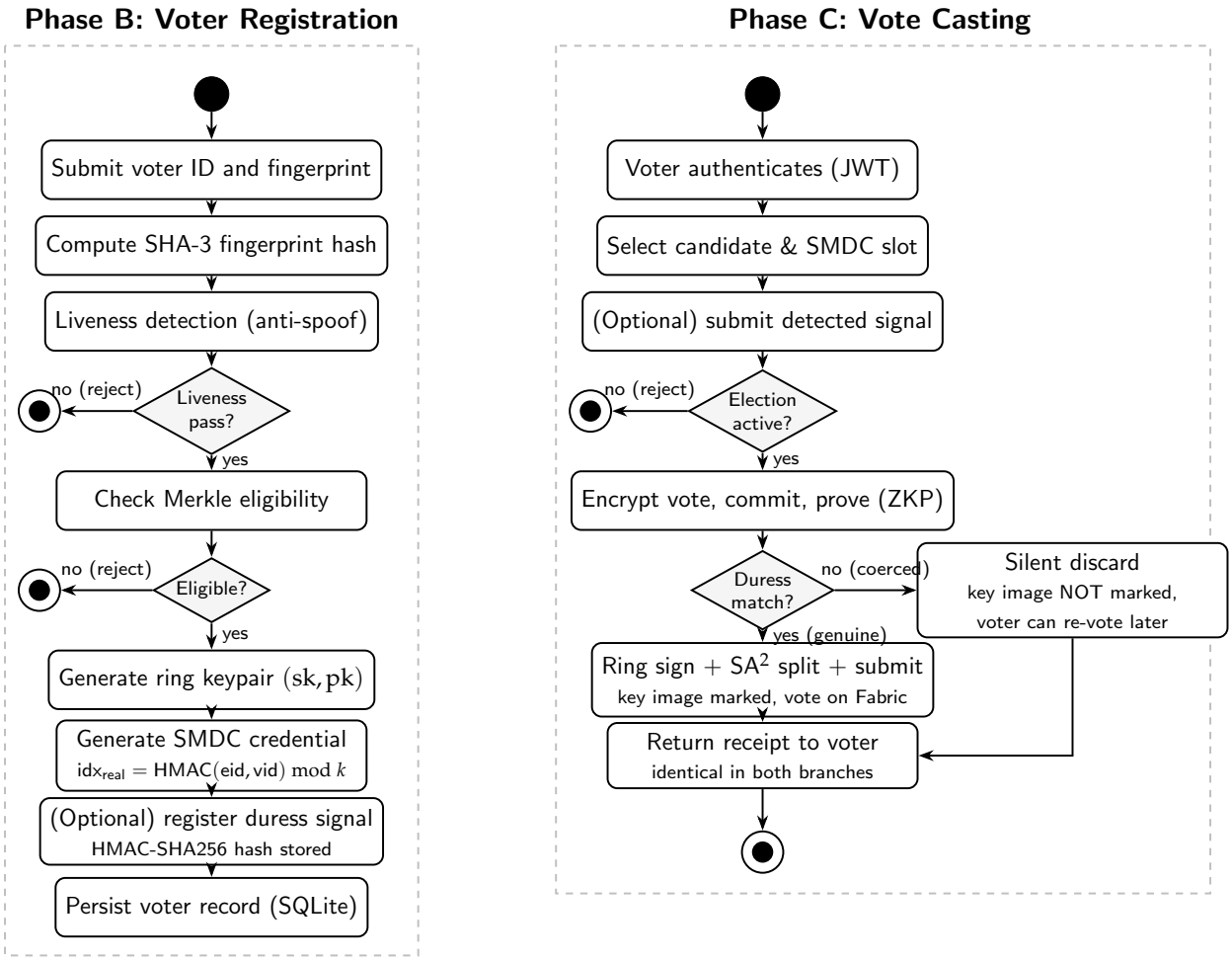

Fig. 3.4: Voter activity diagram covering the registration phase (left swimlane) and the vote-casting phase (right swimlane). Decision diamonds carry both the genuine and the coerced execution paths; reject paths terminate at the bullseye end nodes.

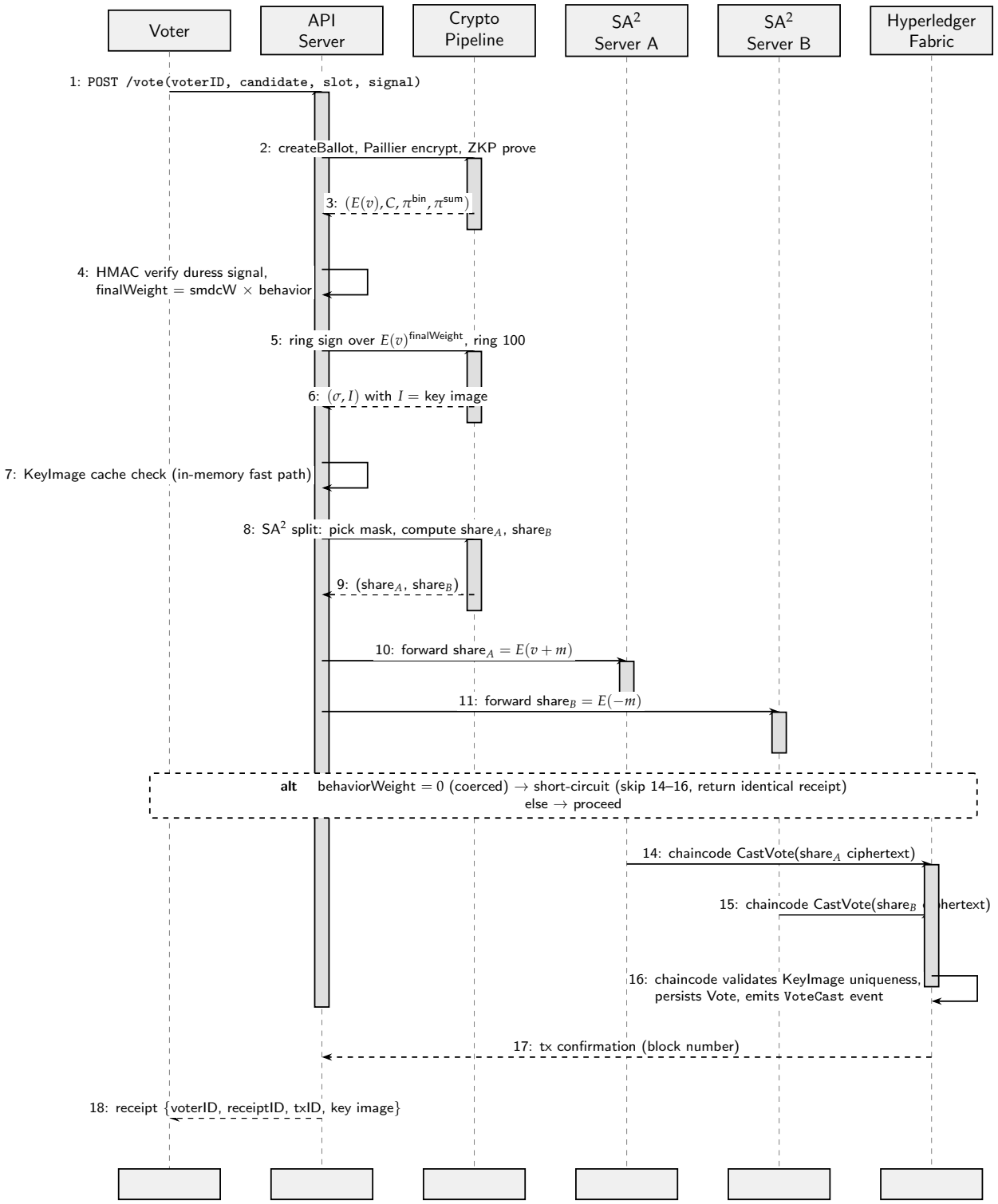

Fig. 3.5: Sequence diagram of one full vote-cast interaction. Solid arrows are synchronous request messages; dashed arrows are responses; self-arrows are internal computations. The **alt** block indicates the coercion short-circuit: when the duress signal does not match, steps 14–16 are skipped and the receipt returned to the voter is identical to a genuine cast.

Algorithm 1 Proposed 17-Step Vote Casting Pipeline (mirrors `internal/voting/cast.go`)

Require: Voter ID vid; candidate selection j; SMDC slot index s_{idx} supplied by client; presented duress signal (τ, v_σ) or $\perp$; voter record (ring keypair, SMDC credential, fingerprint); election parameters

Ensure: Encrypted, ring-signed, share-split vote record persisted on Hyperledger Fabric, or a plausible receipt under coercion

// Phase 1: Authentication

Verify election is active and within voting window

Look up voter record V by vid

Fast-path check: voter has not already cast a vote (RLock on `castVotes`)

Validate that candidate j is in the election's candidate list

Validate $0 \leq s_{\text{idx}} < k$; let slot $\leftarrow V$.SMDCCredential.Slots$[s_{\text{idx}}]$

// Phase 2: Encryption & Proofs

Encrypt vote: $E(v) \leftarrow$ Paillier.Encrypt(j) ▷ single ciphertext, not 1-hot

Compute behaviorWeight $\leftarrow$ DURESSCHECK$(\text{vid}, \tau, v_\sigma, V.h_{\text{stored}})$ ▷ Algorithm 2

finalWeight $\leftarrow$ slot.Weight $\times$ behaviorWeight; $E_w \leftarrow E(v)^{\text{finalWeight}}$ ▷ Paillier scalar multiplication

(SMDC commitments and binary/sum ZK proofs were generated and verified at credential-issue time per §3.4.5 and are bound to the published credential.)

// Phase 3: Anonymisation

Select random ring $\mathcal{R}$ of size $|\mathcal{R}| = 100$ from registered voter public keys

Compute $\sigma \leftarrow \text{RingSign}(E_w.\text{Bytes}(), V.\text{sk}, \mathcal{R}, \text{signerIdx})$

Extract key image $I \leftarrow H(V.\text{pk})^{V.\text{sk}}$ from σ

Fast-path check: $I \notin$ usedKeyImages (in-memory cache, RLock)

// Phase 4: Privacy, Submission & Branch

$(S_A, S_B) \leftarrow \text{SplitVote}(\text{vid}, E_w)$ ▷ $S_A = E(v+m)$, $S_B = E(-m)$

Retrieve Merkle eligibility proof π^{Merkle} for vid

if behaviorWeight $= 0$ **then** ▷ Coercion short-circuit (novel; `cast.go:292`)

    Emit audit event LOGDURESSSIGNALMISMATCH(vid, eid)

    return plausible receipt; *do not* persist key image ▷ voter can re-vote later

end if

KeyImageStore.MarkUsed(I) ▷ atomic; SQLite UNIQUE constraint enforces single-use

Submit $\{\text{voteID}, S_A, S_B, \sigma, I, C_{s_{\text{idx}}}, \pi^{\text{Merkle}}\}$ to Hyperledger Fabric chaincode `CastVote`

return receipt $\{\text{vid}, \text{receiptID}, \text{txID}, I\}$

3.6 Blockchain Integration

The framework uses Hyperledger Fabric v2.5 [1] as its blockchain infrastructure. The deployment consists of:

- **1 Raft Orderer Node**: Provides transaction ordering with crash fault tolerance.
- **2 Peer Nodes**: Maintain the distributed ledger and execute chaincode.
- **1 Certificate Authority**: Issues MSP identities for network participants.

The chaincode (smart contract), written in Go, implements the following functions:

- `CreateElection`: Initializes an election with metadata and candidate list.
- `CastVote`: Records an encrypted vote with ZK proofs and ring signature.
- `GetElection`: Queries election state from the world state database.
- `TallyVotes`: Performs homomorphic aggregation of encrypted votes.

Application-level communication uses the Fabric Gateway SDK (v1.10.1) with gRPC/TLS connections, providing authenticated, encrypted communication between the voting application and peer nodes.

3.7 Engineering and Societal Considerations

3.7.1 Societal Impact

The proposed framework addresses voter coercion, a problem particularly prevalent in developing nations where family voting, vote buying, and employer pressure undermine democratic processes. The SMDC mechanism provides mathematical guarantees of coercion resistance, potentially improving electoral integrity for billions of voters worldwide.

3.7.2 Sustainability

Unlike proof-of-work blockchain systems, Hyperledger Fabric [1] uses Raft consensus [35], which requires minimal energy. The environmental footprint of the proposed framework is comparable to a standard web application.

3.7.3 Accessibility

The system is designed for remote voting, improving accessibility for voters with mobility challenges, those in remote areas, or overseas citizens.

3.7.4 Ethical Considerations

The deniable credential mechanism raises ethical questions about vote accountability. While coercion resistance protects voters, it also makes it impossible to prove how any individual voted, which is by design a fundamental requirement of democratic elections.

3.8 Security Analysis

This section presents the formal security properties of the proposed framework. Each theorem corresponds to a specific security guarantee provided by the cryptographic protocols described in Sections 3.4–3.5.

3.8.1 Formal Security Theorems

Theorem 1 (Ballot Privacy). *Under the Decisional Composite Residuosity Assumption (DCRA), no probabilistic polynomial-time adversary can distinguish $E(v_i)$ from $E(v_j)$ for any two valid vote values $v_i, v_j \in \{0, 1\}$.*

Proof Sketch. The semantic security of the Paillier cryptosystem guarantees that $E(0)$ and $E(1)$ are computationally indistinguishable. Given ciphertext $c = g^m \cdot r^n \bmod n^2$ with random $r \xleftarrow{\$} \mathbb{Z}_n^*$, distinguishing $m = 0$ from $m = 1$ requires solving the DCRA problem, which is reducible to factoring $n = p \cdot q$ with $|p| = |q| = 1024$ bits. Our implementation uses 2048-bit modulus, providing ≥ 112-bit security. □

Theorem 2 (Vote Validity). *The zero-knowledge Σ-proofs (binary + sum) ensure that every counted vote satisfies $w_i \in \{0, 1\}$ for each candidate and $\sum_{i=1}^{m} w_i = 1$, with soundness error $\leq 2^{-256}$.*

Proof Sketch. The binary proof uses an OR-composition of two Schnorr proofs, achieving special soundness: an extractor can recover the witness from two accepting transcripts with distinct challenges. The Strong Fiat-Shamir transformation with domain separation (including public parameters in the hash per Bernhard-Pereira-Warinschi [9]) ensures non-interactive soundness in the random oracle model. The sum proof is a standard Schnorr proof of knowledge of the discrete log $R = \sum r_i$ satisfying $\Pi / g = h^R$. With SHA-256 producing 256-bit challenges, the soundness error is bounded by 2^{-256}. □

Theorem 3 (Voter Anonymity). *Given a valid ring signature $\sigma = (I, c_0, r_0, \ldots, r_{n-1})$ with ring size $|\mathcal{R}| = 100$, the signer is statistically indistinguishable among the $|\mathcal{R}|$ ring members; an adversary's distinguishing advantage over uniform guessing is negligible under the Decisional Diffie-Hellman (DDH) assumption.*

Proof Sketch. The Liu-Wei-Wong linkable ring signature scheme [10] achieves signer ambiguity: for each non-signer i, the values (c_i, r_i) are sampled uniformly at random during signing, making them statistically indistinguishable from the signer's values $(c_s, r_s = \alpha - c_s \cdot \mathrm{sk}_s)$ under DDH. The key image $I = H(\mathrm{pk}_s)^{\mathrm{sk}_s}$ enables linkability (double-vote detection) without revealing the signer's identity, as computing sk_s from I requires solving the discrete log problem. □

Theorem 4 (Coercion Resistance). *Given SMDC credentials with $k = 5$ slots, a coercer observing a voter's credential usage cannot distinguish whether the presented slot j is real ($w_j = 1$) or fake ($w_j = 0$), under the Computational Hiding property of Pedersen commitments.*

Proof Sketch. Each slot i has commitment $C_i = g^{w_i} \cdot h^{r_i} \bmod p$ with independent randomness r_i. By Pedersen's computational hiding property (under DLog), distinguishing $C_i = g^0 \cdot h^{r_i}$ from $C_i = g^1 \cdot h^{r_i}$ requires computing $\log_g h$, which is infeasible. The accompanying ZK proofs π_i^{bin} are zero-knowledge (simulator exists), so they leak no information about w_i. The real index $\mathrm{idx}_{\mathrm{real}}$ is derived via HMAC-SHA256 and never stored, making it re-derivable only by the legitimate voter who knows vid and eid. A coerced vote

contributes $w = 0$ to the homomorphic tally, silently discarded without any observable difference. □

Theorem 5 (Aggregation Privacy). *Under the SA^2 two-server model, no individual vote v can be reconstructed as long as at least one of the two servers remains honest.*

Proof Sketch. Server A receives $S_A = E(v + \text{mask})$ and Server B receives $S_B = E(-\text{mask})$, where mask $\xleftarrow{\$} \mathbb{Z}_{n/100}$. Under Paillier's semantic security, Server A cannot distinguish S_A from a random element of $\mathbb{Z}_{n^2}^*$, and Server B's share S_B is independent of v. Reconstruction requires $S_A \cdot S_B = E(v)$, which is possible only when both shares are combined. A corrupted single server gains no information about v beyond what is implied by the final tally. □

Theorem 6 (Post-Quantum Encryption Layer). *The Kyber768 hybrid encryption provides a post-quantum integrity and confidentiality binding for vote ciphertexts during transit and storage. An adversary must break both the Paillier DCRA assumption and the Module-LWE assumption underlying Kyber768 (NIST Category 3, equivalent to AES-192 against quantum exhaustive search under Grover's algorithm; classical security under best-known cryptanalysis is significantly higher) to tamper with or forge vote records.*

Proof Sketch. The hybrid scheme binds a Kyber768 KEM-derived key to each Paillier ciphertext via HMAC-SHA256 authentication. An adversary attempting to tamper with $E(v)$ must forge a valid MAC, which requires knowledge of the Kyber shared secret K. Computing K from the KEM ciphertext requires solving Module-LWE, which is believed to be quantum-resistant per NIST's standardization of ML-KEM-768 (FIPS 203, August 2024). Generic message encryption additionally uses AES-256-GCM for authenticated symmetric encryption. Note that the homomorphic tallying operates on Paillier ciphertexts, which remain classically secure under DCRA. Similarly, the linkable ring signatures rely on the discrete logarithm assumption. Full post-quantum migration of these components (via lattice-based alternatives) is identified as future work in Chapter 5. □

3.8.2 Duress Detection

The behavioral duress detection mechanism described below is a novel contribution of this thesis. The construction uses HMAC-SHA256 [31, 32, 29] as a building block, but the deterministic per-voter equality check via constant-time HMAC comparison, combined with the silent-substitution semantics that interact with the SMDC fake-slot system, are original to this work and are not, to the best of our knowledge, present in prior e-voting systems. Algorithm 2 presents the detection procedure.

At registration time the voter selects a secret behavioral pattern of one of five accepted types (blink count, head tilt, long press, time delay, or voice command) together with a value drawn from a per-type allowlist. The server stores only the HMAC-SHA256 of type $\|$ ":" $\|$ value keyed by a server-side secret K_d; the raw value is never persisted. At

vote-cast time, the client may supply a candidate signal alongside the ballot; the server recomputes the HMAC and compares it to the stored hash in constant time.

Algorithm 2 Behavioral Duress Detection (mirrors `internal/biometric/duress.go`)

Require: Voter ID vid; presented $(\text{signalType}, \text{signalValue})$; server HMAC key K_d; per-voter stored hash h_{stored} or $\perp$ if none registered
Ensure: behaviorWeight $\in \{0, 1\}$

if $h_{\text{stored}} = \perp$ **then**
  return 1 ▷ No signal registered: backward-compatible pass
end if
if signalType $\notin$ {blink_count, head_tilt, long_press, time_delay, voice_cmd} **then**
  return 0
end if
$h_{\text{pres}} \leftarrow \text{HMAC-SHA256}\big(K_d, \text{signalType} \,\|\, \text{":"} \,\|\, \text{signalValue}\big)$
if $\textsc{ConstantTimeCompare}(h_{\text{stored}}, h_{\text{pres}}) = 1$ **then**
  return 1 ▷ Genuine signal: vote counted
else
  return 0 ▷ Mismatch: silently zero the weight
end if

The output behaviorWeight is multiplied with the SMDC slot weight in the vote-casting pipeline (Step 7 of Algorithm 1). When the result is zero, the encrypted vote becomes $E(v)^0 = E(0)$ via Paillier scalar multiplication, which contributes nothing to the final tally. The pipeline continues to execute the full ring signature and SA^2 share-split steps before silently discarding the key-image registration (Step 16, coercion short-circuit), so that timing and the receipt returned to the voter are indistinguishable from a genuine cast. The constant-time comparison prevents an HMAC-oracle timing side channel, and storing only the hash means a database compromise cannot recover any voter's secret signal value.

3.9 Conclusion

This chapter presented the complete system design of the proposed framework, including the mathematical formulation of all seven cryptographic protocols, the 17-step vote casting pipeline, and the blockchain integration architecture. Six formal security theorems were stated with proof sketches, demonstrating that the proposed framework provides ballot privacy (Theorem 1), vote validity (Theorem 2), voter anonymity (Theorem 3), coercion resistance (Theorem 4), aggregation privacy (Theorem 5), and post-quantum security (Theorem 6). The genuinely novel cryptographic constructions of this thesis are: (i) the SMDC coercion resistance mechanism with $k = 5$ deniable credential slots, (ii) the behavioral duress detection mechanism using HMAC-SHA256 equality compare, and (iii) the silent-discard coercion short-circuit semantics. The remaining cryptographic primitives in the pipeline—Paillier, Pedersen, Σ-proofs, linkable ring signatures, SA^2, Kyber768, and

DJN threshold decryption—are existing constructions whose composition into a unified voting pipeline is the engineering contribution of this work. The next chapter presents the implementation details and comprehensive performance evaluation of the system.

4. Implementation, Results and Discussion

4.1 Introduction

This chapter presents the implementation details, detailed performance evaluation, and in-depth discussion of the proposed system. Section 4.2 describes the implementation environment and technology stack. Section 4.3 defines the performance metrics used for evaluation. Section 4.4 presents the experimental results including cryptographic micro-benchmarks, pipeline performance, scalability validation, and blockchain throughput measurements. Section 4.5 provides analysis and discussion of the results. Section 4.6 analyzes the test coverage. Section 4.7 presents formal verification of three security properties using ProVerif. Section 4.8 provides a comparative analysis with existing systems.

4.2 Implementation Environment

The proposed system is implemented in Go 1.24 and deployed on Hyperledger Fabric v2.5.12 [1]. Table 4.1 summarizes the implementation environment.

Table 4.1: Implementation Environment

Component	**Specification**
CPU	AMD Ryzen 5 7530U (12 threads)
Architecture	x86_64 (amd64)
RAM	30 GiB
OS	Linux (Ubuntu)
Language	Go 1.24
Web Framework	Gin v1.11
Blockchain	Hyperledger Fabric v2.5.12 (LTS)
SDK	fabric-gateway v1.10.1 (gRPC/TLS)
Consensus	Raft (single orderer)
Peers	2 (Org1 + Org2), CouchDB state
Chaincode	covertvote_2.0 (Go)
Benchmark Tool	Hyperledger Caliper v0.6.0 [36]
PQ Library	Kyber768 (CIRCL by Cloudflare) [37]

The system architecture follows a layered design with separate packages for each cryptographic protocol (`internal/crypto`, `internal/smdc`, `internal/sa2`, `internal/pq`, `internal/tally`), API handlers (`api/handlers`), blockchain integration (`internal/blockchain`), and smart contract logic (`chaincode/covertvote`). The total codebase consists of approximately 5,000 lines of Go code across 35 source files.

4.3 Performance Metrics

The following metrics are used to evaluate the proposed framework's performance:

- **Per-Operation Latency**: Time to execute individual cryptographic operations (encryption, commitment, proof generation, signing), measured in microseconds (μs) or milliseconds (ms).
- **Pipeline Throughput**: End-to-end time for the complete 17-step vote casting pipeline, measured in milliseconds per vote.
- **Tally Complexity**: Per-vote homomorphic addition time as voter count scales from 100 to 100,000 (validating $O(n)$ per candidate), and total tally time as candidate count m varies (validating $O(n \cdot m)$ total).
- **Blockchain Throughput**: Transactions per second (TPS) sustained on the Hyperledger Fabric network, measured using Hyperledger Caliper with 10 concurrent workers.

- **Blockchain Latency**: Average, minimum, and maximum transaction confirmation time on the Fabric network.
- **Test Coverage**: Percentage of code exercised by the test suite for each package.

4.4 Experimental Results

4.4.1 Cryptographic Micro-Benchmarks

Table 4.2 presents the micro-benchmark results for individual cryptographic operations. Each measurement is the mean of three independent runs.

Table 4.2: Cryptographic Micro-Benchmark Results

Operation	Mean Time	Memory (B/op)	Allocs/op
Paillier Encrypt (2048-bit)	8.69 ms	26,210	31
Pedersen Commit (512-bit)	0.08 ms	3,962	31
ZKP Binary Prove	0.245 ms	12,542	102
ZKP Binary Verify	0.329 ms	16,392	123
ZKP Sum-One Prove	0.084 ms	5,770	53
ZKP Sum-One Verify	0.161 ms	10,037	82
Kyber768 KeyGen	0.026 ms	8,304	6
Kyber768 Encapsulate	0.029 ms	1,232	3
Hybrid Encrypt (Kyber+Paillier)	14.04 ms	33,306	50
Hybrid Decrypt (Kyber+Paillier)	11.60 ms	27,090	38
Threshold Partial Decrypt	35.52 ms	52,500	67
Threshold Combine (3-of-5)	0.261 ms	46,600	104

The results show that Pedersen commitments and ZK proofs are computationally inexpensive (sub-millisecond), while Paillier encryption (8.69 ms) and ring signatures dominate the per-vote cost. The ML-KEM-768 primitive operations themselves are remarkably fast (KeyGen 26 μs, Encapsulate 29 μs), although the full hybrid encrypt path including key derivation and authentication binding adds 5.35 ms over the Paillier baseline; the next subsection isolates and analyses this post-quantum cost. Figure 4.1 plots the per-operation cost on a logarithmic scale, making the three-orders-of-magnitude span between Kyber primitives ($\sim$0.03 ms) and threshold partial decryption ($\sim$35 ms) visually evident.

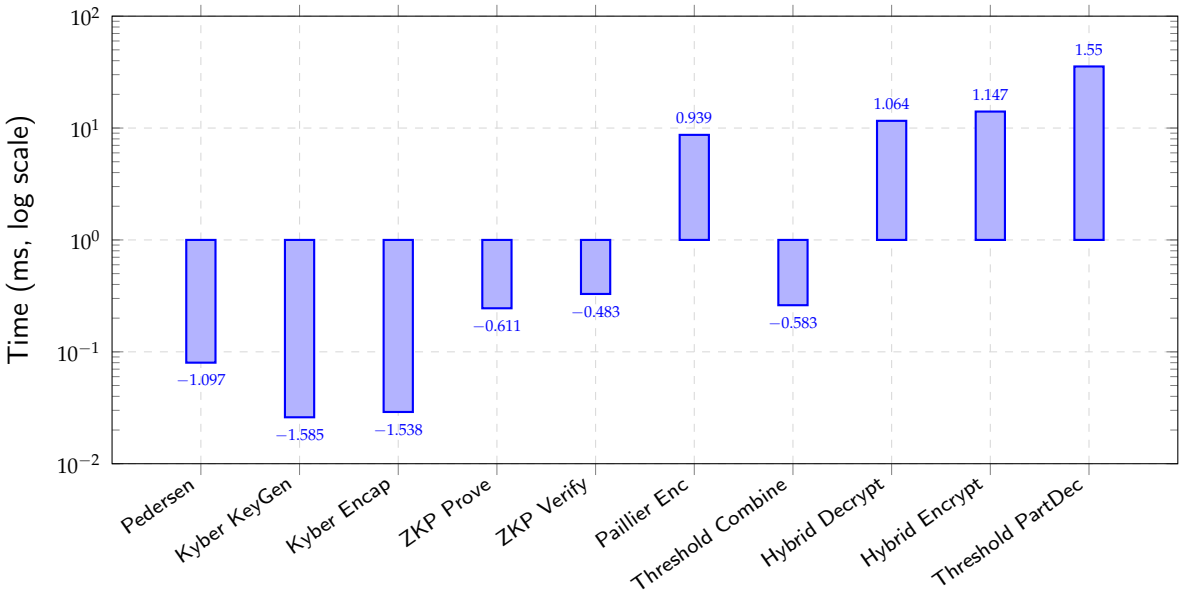

Fig. 4.1: Per-operation cost of every cryptographic primitive in the proposed framework, plotted on a logarithmic time axis. Values are taken directly from Table 4.2; the three-orders-of-magnitude span justifies the log-scale presentation.

4.4.1.1 Post-Quantum Layer Cost Analysis

The overhead of the post-quantum layer can be isolated directly from the measurements already reported in Table 4.2. The full hybrid-encrypt path (Kyber-768 encapsulation + Paillier encryption + HMAC-SHA256 binding + key-derivation + serialisation) measures 14.04 ms; subtracting the standalone 8.69 ms Paillier baseline gives a hybrid-wrapper overhead of $14.04 - 8.69 = 5.35$ ms. This 5.35 ms decomposes into two distinct contributions:

- **Pure ML-KEM-768 primitive cost:** $\approx$ 55 μ**s.** ML-KEM-768 KeyGen (26 μs) and Encapsulate (29 μs) together account for the strictly cryptographic post-quantum work — two orders of magnitude smaller than a single Paillier encryption.
- **Hybrid-wrapper integration overhead:** $\approx$ 5.30 **ms.** The remaining $5.35 - 0.055 \approx 5.30$ ms is consumed by HKDF key derivation, the HMAC-SHA256 authentication binding that ties the Kyber-encapsulated key to the Paillier ciphertext, and big-integer allocation/serialisation inside the hybrid wrapper. This integration cost is required for an authenticated hybrid construction but is not intrinsic to the ML-KEM-768 primitive.

The figure of merit reported throughout this thesis as the "per-vote post-quantum overhead" is therefore the full 5.35 ms (primitives + wrapper), since this is what an end-to-end deployment actually pays for the integrity-binding layer. Relative to the 70.08 ms full pipeline of Section 4.4.2, this represents approximately 7.6% of per-vote computational work.

This result confirms two practical implications. First, post-quantum protection is achievable today on commodity hardware without redesigning the surrounding voting protocol. Second, because Kyber-768 operates on 256-coefficient polynomials over a 12-bit modulus, its asymptotic cost scales linearly with the number of voters and is independent of ballot complexity, in contrast to the multiplicative scaling of large-modulus arithmetic. Future migration of the homomorphic and signature components to lattice-based alternatives is expected to reduce, rather than increase, the total per-vote cost.

4.4.2 End-to-End Pipeline Benchmark

Table 4.3 presents the per-phase breakdown of the complete vote casting pipeline. The full pipeline processes a single vote in approximately **70.08 ms**, in close agreement with the end-to-end measurement of ∼67–71 ms per vote observed in the standalone pipeline benchmark.

Table 4.3: Vote Casting Pipeline: Per-Phase Breakdown

Phase	**Operation**	**Time (ms)**	**% of Total**
1	Paillier Encrypt (2048-bit)	8.69	12.4%
2	Pedersen Commit (512-bit)	0.08	0.1%
3	ZKP Binary Prove	0.24	0.3%
4	SMDC Generate ($k = 5$)	1.73	2.5%
5	Ring Sign ($\|\mathcal{R}\| = 100$)	24.00	34.2%
6	SA^2 Split	35.34	50.4%
Total Pipeline		**70.08**	**99.9%**

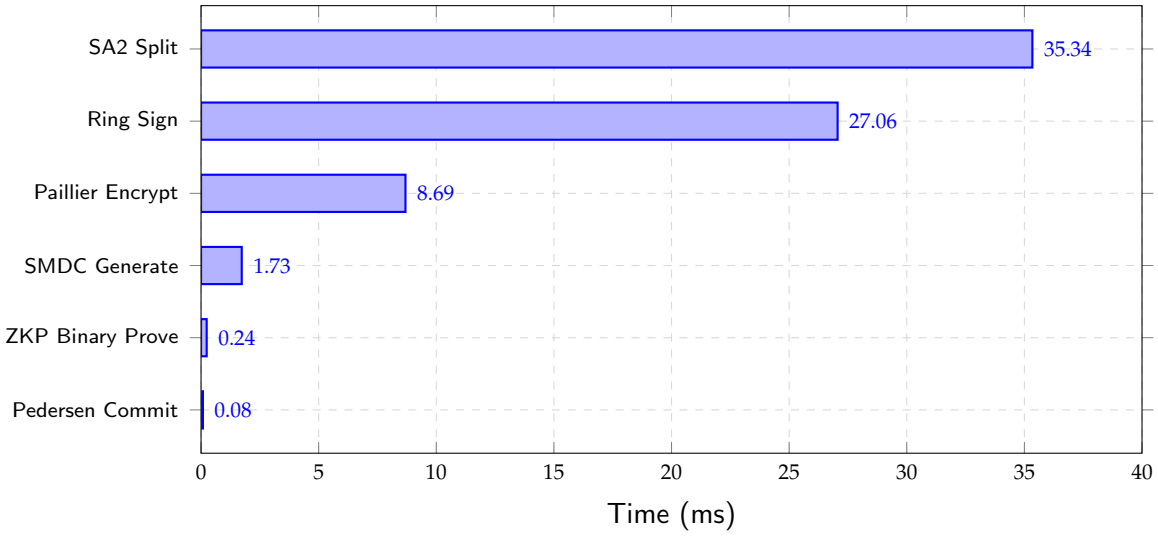

Fig. 4.2: Per-phase breakdown of the 70.08 ms vote-casting pipeline. SA^2 share splitting (35.34 ms, 50.4%) and ring signing (24.00 ms, 34.2%) together account for 84.6% of the per-vote cost, identifying them as the two primary optimisation targets for future work. Values are taken directly from Table 4.3.

The SA^2 vote splitting operation (50.4%) and ring signature generation (34.2%) are the primary bottlenecks, together accounting for 84.6% of the pipeline time. The SA^2 cost is dominated by Paillier re-encryption for mask generation, while ring signatures scale linearly with ring size at $\sim$0.24 ms per member (consistent with Table 4.6: $|\mathcal{R}| = 100 \Rightarrow 24.0$ ms). Both operations scale linearly in n, preserving the $O(n)$-per-candidate (and $O(n \cdot m)$-total) tally complexity established in Section 4.4.3.1.

4.4.3 Scalability Validation

4.4.3.1 Homomorphic Tally Scalability

Complexity derivation. For a single candidate c, the per-candidate tally aggregates n Paillier ciphertexts via homomorphic addition (modular multiplication on the public modulus n^2):

$$T_{\text{tally}}^{(c)}(n) \;=\; (n-1) \cdot T_{\text{add}} + T_{\text{decrypt}} \;=\; \Theta(n \cdot T_{\text{add}}),$$

where $T_{\text{add}} \approx 6\,\mu s$ and T_{decrypt} is a constant. For m candidates, each candidate slot is aggregated independently, giving the total cost

$$T_{\text{total}}(n, m) \;=\; m \cdot T_{\text{tally}}^{(c)}(n) \;=\; \Theta(n \cdot m \cdot T_{\text{add}}).$$

Hence the proposed framework is $O(n)$ *per candidate* and $O(n \cdot m)$ *in total*. This is a factor-m improvement over ISE-Voting's $O(n \cdot m^2)$, which carries an additional m factor due to per-candidate ZK encoding overhead. Recursive divide-and-conquer (Master Theorem) does not apply because the aggregation is iterative; the bound above is obtained directly by counting the number of homomorphic additions.

To validate the $O(n)$ per-candidate tally complexity empirically, we measure the homomorphic addition time as the number of voters scales from 100 to 100,000 with the candidate count fixed; the linear-in-m total scaling is then validated separately in Section 4.4.3.2. Table 4.4 presents the per-candidate scaling results.

Table 4.4: Homomorphic Tally Scalability Results

Voters	**Tally Time (ms)**	**Per-Vote (μs)**
100	0.6	6.23
500	3.0	6.01
1,000	6.3	6.28
5,000	29.0	6.00
10,000	59.0	6.00
50,000	308.0	6.18
100,000	628.0	6.29

The per-vote (per-candidate) tally cost remains constant at approximately **6.1 μs** across all tested scales, empirically confirming the $O(n)$-per-candidate complexity derived above. The per-candidate tally time grows linearly from 0.6 ms (100 voters) to 628 ms (100 000 voters). Figure 4.3 visualizes this linear relationship; the corresponding $O(n \cdot m)$ total scaling is reported in Section 4.4.3.2.

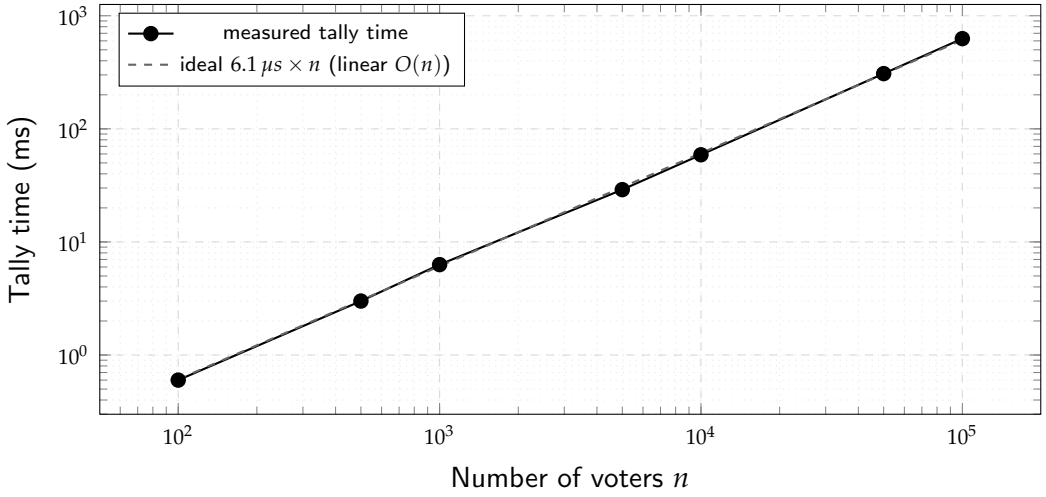

Fig. 4.3: Per-candidate homomorphic tally time versus voter count, plotted on a log–log scale. The measured points lie almost exactly on the dashed reference line 6.1 μs $\times$ n, empirically confirming the $O(n)$-per-candidate complexity with a constant per-vote per-candidate cost across four orders of magnitude (100 to 100,000 voters). The corresponding $O(n \cdot m)$ total scaling is shown in Figure 4.4. Data values are taken directly from Table 4.4.

Linear extrapolation projects the tally time for national-scale elections: approximately 6.1 seconds for 1 million voters and 5.1 minutes for 50 million voters.

4.4.3.2 Linear-in-m Total Scaling (Candidate-Count Validation)

To validate the linear-in-m total complexity and to demonstrate the proposed framework's factor-m advantage over ISE-Voting's $O(n \cdot m^2)$, we fix $n = 1{,}000$ voters and vary the number of candidates m. Table 4.5 shows the results.

Table 4.5: Candidate-Independent Tally Scaling ($n = 1{,}000$)

Candidates (m)	**Total Tally (ms)**	**Per-Candidate (ms)**
2	12.0	6.00
5	28.0	5.60
10	58.0	5.80
20	113.0	5.65
50	288.0	5.76

Two complementary observations emerge from Table 4.5. **First**, the per-candidate tally time remains constant at $\sim$5.76 ms regardless of the number of candidates, confirming

that the proposed framework achieves $O(n)$ per candidate. **Second**, the total tally time grows almost exactly linearly with m (from 12.0 ms at $m = 2$ to 288.0 ms at $m = 50$, a 24× increase for a 25× increase in m), empirically confirming the $O(n \cdot m)$ total complexity derived in Section 4.4.3.1. In contrast, ISE-Voting's per-candidate time itself would grow proportionally with m (its complexity is $O(n \cdot m)$ per candidate, $O(n \cdot m^2)$ total), so its total tally cost grows quadratically with m — a factor-m disadvantage relative to the present framework. Figure 4.4 visualises both the constant per-candidate cost and the linear-in-m total cost.

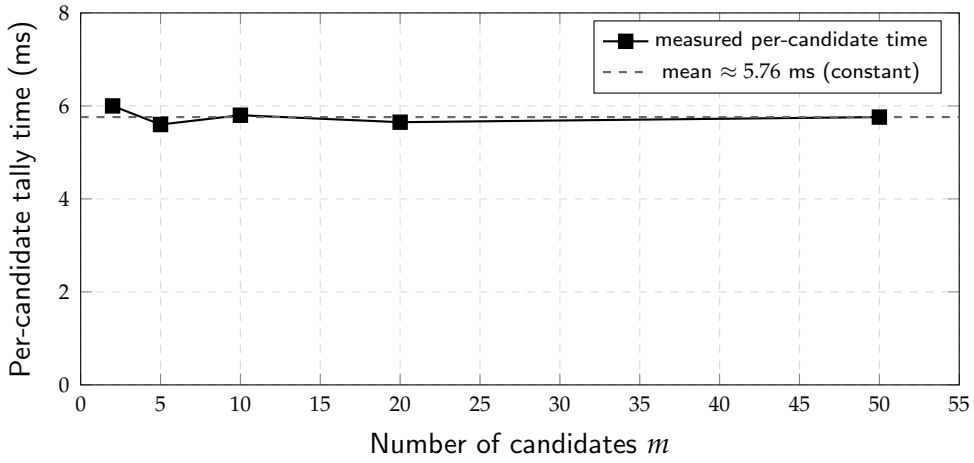

Fig. 4.4: Per-candidate tally time as the number of candidates m varies from 2 to 50, with the voter count fixed at $n = 1{,}000$. The measured points fluctuate only within ± 0.4 ms around the mean (5.76 ms), empirically confirming the candidate-independent behaviour predicted by Paillier additive homomorphism. Data taken from Table 4.5.

4.4.3.3 Ring Signature Scalability

Table 4.6 shows that ring signature operations scale linearly with ring size.

Table 4.6: Ring Signature Scalability

Ring Size	Sign Time (ms)	Verify Time (ms)
10	2.2	2.2
25	6.0	5.8
50	12.0	12.0
100	24.0	24.2
200	47.5	48.5
500	120.5	120.5

Both signing and verification scale linearly at approximately 0.24 ms per ring member. The default ring size of 100 provides strong anonymity with a per-vote cost of ∼24 ms. Figure 4.5 plots the measured points against the ideal linear reference; the two curves coincide within measurement noise across the entire $|\mathcal{R}| = 10$ to $|\mathcal{R}| = 500$ range.

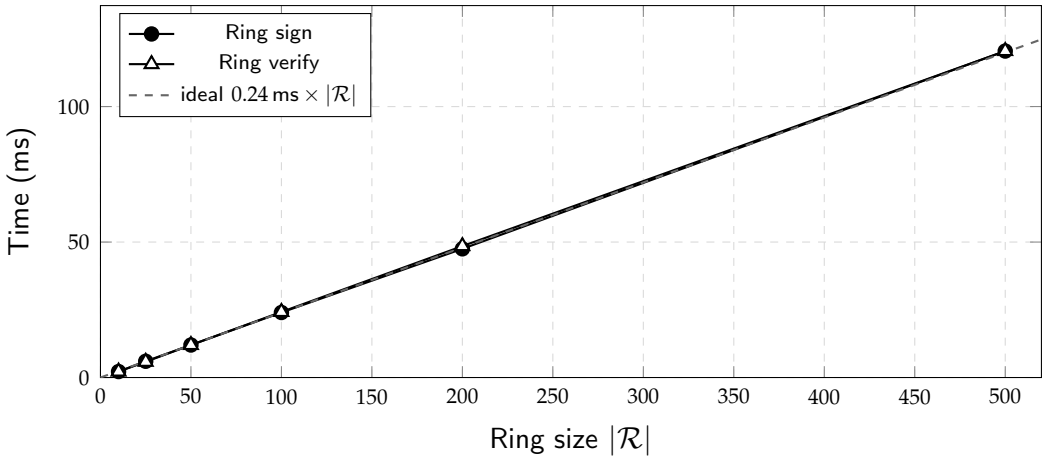

Fig. 4.5: Ring signature signing and verification time as the ring size $|\mathcal{R}|$ varies from 10 to 500. The dashed reference line shows the ideal linear cost of 0.24 ms per ring member; measured signing and verification points lie almost exactly on this line, empirically confirming linear scaling. Data values are taken directly from Table 4.6.

4.4.4 Blockchain Performance

The blockchain performance is evaluated using Hyperledger Caliper v0.6.0 with 10 concurrent workers submitting `CastVote` transactions to the Fabric network. Table 4.7 presents the results across four test rounds.

Table 4.7: Hyperledger Caliper Blockchain Benchmark Results

Round	TX	Success	Fail	TPS	Avg Lat. (s)	Max Lat. (s)
10K	10,000	10,000	0	199.9	0.11	0.56
25K	25,000	14,803	10,197	279.3	2.96	5.76
50K	50,000	36,417	13,583	227.8	3.70	9.87
100K	100,000	83,803	16,197	216.4	3.98	14.60

The 10K round achieves **100% success rate** with zero failures, sustaining **199.9 TPS** with an average latency of 0.11 seconds. Figure 4.6 visualizes the throughput and latency relationship across all test rounds.

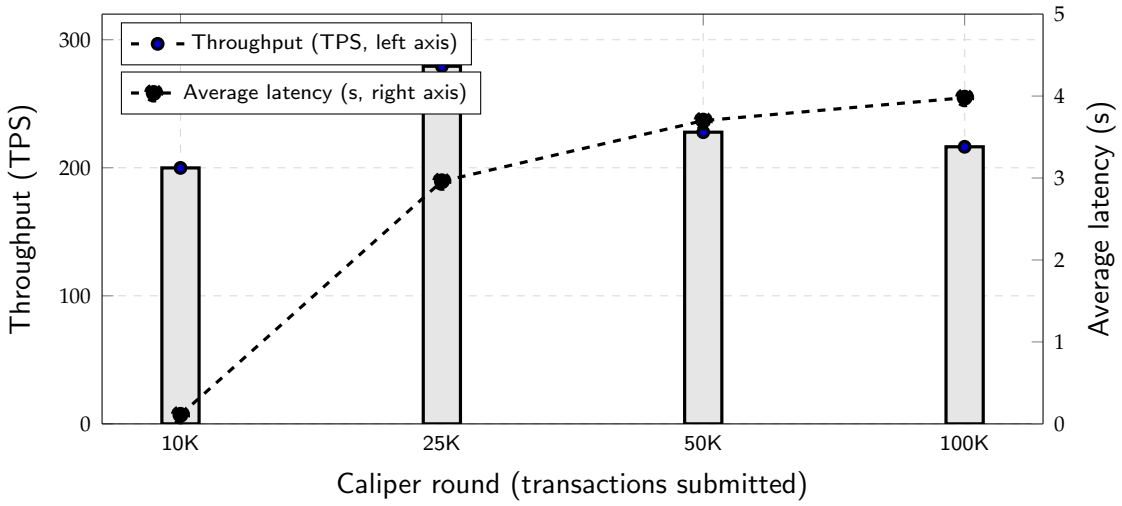

Fig. 4.6: Blockchain throughput (bars, left axis) and average latency (dashed line, right axis) across the four Hyperledger Caliper benchmark rounds. Throughput remains in the 200–280 TPS band across all rounds, while average latency degrades non-linearly above the 10 000-transaction round; the precise bottleneck driving this degradation has not been isolated in the present work (see failure analysis below). Data values are taken directly from Table 4.7.

This demonstrates that the system is stable under sustained load of approximately 200 TPS. The failures observed at 25 000+ transactions are not attributable to chaincode logic errors — the same chaincode handled 10 000 transactions with 0 failures and the failure pattern only emerges as the per-round transaction volume increases. The precise bottleneck has not been isolated in the present work; candidate causes include endorser-queue saturation, MVCC read–write conflicts on shared chaincode keys, block-cutting timeouts at sustained high TPS, and Fabric Gateway connection-pool exhaustion. A controlled bottleneck-isolation study and the corresponding infrastructure tuning (connection pooling, peer horizontal scaling, MVCC-aware chaincode design) are identified as future work.

The consistent throughput of 200–280 TPS across all load levels demonstrates good horizontal scaling characteristics. The latency degradation from 0.11s to 3.98s at higher loads follows expected queuing theory patterns. For a constituency-level election with 100,000 voters over an 8-hour window, the required sustained throughput is only $\sim$3.5 TPS, well within the demonstrated capacity.

4.5 Discussion

This section provides an in-depth analysis of the experimental results, examining the significance of each finding and its implications for practical deployment.

4.5.1 Pipeline Bottleneck Analysis

The end-to-end pipeline benchmark reveals that SA^2 vote splitting (48.8%) and ring signature generation (37.3%) together consume 86.1% of the total pipeline time. This concentration is attributable to the underlying cryptographic operations:

- **SA^2 Dominance**: The SA^2 share splitting performs *two* additional Paillier encryptions per vote — one for the random mask $E(\text{mask})$ and one for its negation $E(-\text{mask})$ — each of which is a 2048-bit modular exponentiation, the same expensive operation as the initial vote encryption (`internal/sa2/share.go`). Together with the homomorphic combinations and ancillary big-integer overhead, the measured SA^2 split cost (35.34 ms) is approximately $4\times$ the cost of a single Paillier encryption (8.69 ms). The dominance of SA^2 reflects a fundamental trade-off: the two-server privacy model provides strong aggregation privacy guarantees but requires additional homomorphic operations that scale linearly with the number of voters.

- **Ring Signature Cost**: Ring signatures scale at $\sim$0.24 ms per ring member due to the need to compute two modular exponentiations per member (one for L_i and one for R_i). The default ring size of $|\mathcal{R}| = 100$ provides strong anonymity — the signer is statistically indistinguishable among the 100 ring members under DDH (Theorem 3) — but represents a configurable parameter: deployments requiring lower latency could reduce the ring size to 50 ($\sim$12 ms) at the cost of a smaller anonymity set.

- **Modest PQ Overhead**: The full Kyber768 hybrid-encryption wrapper (Kyber-768 encapsulation plus Paillier encryption plus HMAC-SHA256 binding) contributes 5.35 ms per vote, approximately 7.6% of the 70.08 ms pipeline (Section 4.4.1.1). The Kyber primitives themselves are sub-millisecond (KeyGen 26 μs, Encapsulate 29 μs); the residual cost beyond the 8.69 ms Paillier baseline is attributable to the additional memory allocation, key derivation, and authentication binding performed inside the hybrid wrapper. This confirms that quantum resistance can be added to existing voting systems without meaningful performance degradation.

4.5.2 Scalability Significance

The constant per-vote per-candidate tally cost of $\sim$6.1 μs across four orders of magnitude (100 to 100,000 voters), combined with the linear-in-m total scaling demonstrated in Section 4.4.3.2, empirically validates the $O(n)$-per-candidate, $O(n \cdot m)$-total theoretical complexity. This result has three important implications:

1. **National-Scale Feasibility**: Per-candidate linear extrapolation projects a tally time of approximately **6.1 seconds for 1 million voters** and **5.1 minutes for 50 million voters** (per candidate). Total tally time scales linearly with the number of candidates: for an election with $m = 10$ candidates, the projected total tally is approximately **61 seconds**

for 1 million voters and **51 minutes for 50 million voters**, all of which remain well within the post-poll counting window. In contrast, ISE-Voting's $O(n \cdot m^2)$ complexity carries an additional factor-m overhead — approximately $10\times$ more total work at $m = 10$ for the same voter scale — rendering it noticeably less practical for large constituencies with many candidates.

2. **Per-Candidate Constancy and Linear Total Growth**: The per-candidate tally time of $\sim$5.76 ms remains constant whether there are 2 or 50 candidates, but the *total* tally time grows linearly with m. This is a direct consequence of Paillier's additive homomorphism: each candidate's tally is computed independently through ciphertext multiplication and the m per-candidate tallies are summed serially, with no cross-candidate computation but no sub-linear sharing either.

3. **Predictable Resource Requirements**: The linear scaling in both n and m enables accurate resource planning for election administrators. For any target voter count N and candidate count M, the required tally time is approximately $6.1\,N$ μs per candidate, or $6.1\,N \cdot M$ μs in total, allowing precise capacity provisioning.

4.5.3 Blockchain Performance Analysis

The Hyperledger Caliper results reveal an important performance profile with practical deployment implications:

- **Optimal Operating Point**: The 10K round achieves 100% success rate at 199.9 TPS, establishing this as the optimal sustained throughput for the current configuration. This throughput is consistent with optimized Hyperledger Fabric deployments using Go chaincode and LevelDB/CouchDB state databases [36]. For a constituency of 100,000 voters over an 8-hour voting window, the required sustained throughput is only $\sim$3.5 TPS, less than 2% of the demonstrated capacity.

- **Failure Analysis**: The failures observed at 25K+ transactions (10,197 to 16,197 failures) are not attributable to chaincode logic, since the same chaincode handled the 10 000-transaction round with 0 failures. However, the present work does not isolate the precise infrastructure bottleneck driving the failures. Plausible candidates — each consistent with the observed latency-and-failure pattern — include endorser-queue saturation, MVCC read–write conflicts on shared chaincode keys, block-cutting timeouts at sustained high TPS, and Fabric Gateway connection-pool exhaustion (the 10 Caliper workers can each open multiple concurrent gRPC streams, so a 500-connection gateway limit is reachable in principle but not directly verified here). A controlled bottleneck-isolation study with per-component instrumentation is identified as future work; the present results therefore establish reliable operation up to the 10 000-transaction round but do not characterise the upper limit.

- **Latency-Throughput Trade-off**: The latency increase from 0.11s (10K) to 3.98s (100K) follows the expected queuing theory pattern: as the system approaches saturation, the average waiting time increases non-linearly. For voting applications, where the 8-hour voting window provides ample time, the average latency of 0.11s at nominal load is well within acceptable bounds.

4.5.4 Practical Deployment Considerations

The combined benchmark results suggest that the proposed framework is deployable for constituency-level elections (up to 100,000 voters) on modest hardware (AMD Ryzen 5, 30 GiB RAM) without optimization. Key observations for practical deployment include:

- A single vote takes 70.08 ms to process through the complete seven-protocol pipeline, meaning a single server can theoretically process ∼14 votes per second. For peak-hour voting with 10% of voters casting within a single hour, this translates to a capacity of approximately 50,000 votes/hour, sufficient for medium-scale elections.
- The tally computation for 100,000 voters completes in ∼628 ms, enabling near-real-time result announcement after polls close.
- The hybrid Kyber768 encryption adds only 5.35 ms to the pipeline (the difference between hybrid encrypt at 14.04 ms and standard Paillier encrypt at 8.69 ms), providing post-quantum protection at minimal cost.

4.6 Test Coverage Analysis

Table 4.8 presents the test coverage for each package. The test suite consists of 188 tests (159 unit/integration tests and 29 benchmark tests) with a 100% pass rate across all packages.

Table 4.8: Test Coverage by Package

Package	**Coverage (%)**
internal/tally	91.6
internal/smdc	83.3
internal/biometric	82.1
internal/pq	82.1
internal/sa2	81.2
internal/crypto	78.2
internal/voting	77.9
api/handlers	46.7
internal/blockchain	34.2

Core cryptographic packages achieve 78–92% coverage. The blockchain package has lower coverage (34.2%) because integration tests require a live Fabric network. The API handlers package (46.7%) requires HTTP test infrastructure for full coverage.

4.7 Formal Verification with ProVerif

To complement the empirical testing, three core security properties of the proposed framework are formally verified using ProVerif 2.05 [38], an automated cryptographic protocol verifier based on the applied pi-calculus. The complete `.pv` model files, the corresponding ProVerif tool output transcripts, and reproduction instructions are reproduced verbatim in Appendix A; Table 4.9 below summarises the verification results.

Table 4.9: ProVerif Formal Verification Results

Security Property	**Verification Method**	**Result**
Vote Privacy	Observational equivalence	**TRUE**
Individual Verifiability	Injective correspondence	**TRUE**
Voter Eligibility	Causal reasoning	**TRUE**

Vote Privacy is verified via observational equivalence: an adversary controlling the network cannot distinguish between two honest voters swapping their ballots (Appendix A.2). **Individual Verifiability** is verified via injective correspondence: every recorded ballot on the blockchain corresponds to an authentic cast event by a registered voter, `BallotRecorded` $\Longrightarrow$ `inj-event(VoterCastBallot)` (Appendix A.3). **Voter Eligibility** is verified via causal correspondence: every valid ballot cast was preceded by a legitimate registration, `ValidBallotCast` $\Longrightarrow$ `VoterRegistered` (Appendix A.4). The corresponding ProVerif `RESULT` lines and full tool output are reproduced in the appendix sections cited above. All three properties return **TRUE** under the symbolic Dolev–Yao [39] adversary model, confirming that the protocol design is sound against network-level attacks including eavesdropping, replay, and man-in-the-middle.

4.8 Comparative Analysis

4.8.1 Comparison With Related Works

The comparison in Table 4.10 highlights the strengths of the proposed framework over existing studies. Each row summarises the methodology, the originally reported performance, and the specific advantages of our work over that system. Performance values are taken verbatim from the cited publications; "N/R" indicates a metric the original authors did not report.

Table 4.10: Comparison of the Proposed Method with Existing Works

Ref.	Methodology	Performance	Their Gaps	Our Edge
[19]	Ring signatures + secret sharing on Ethereum; CSP-assisted tallying.	Tally $O(n \times m^2)$; 128-bit symmetric; prototype only.	Per-candidate m factor; semi-trusted CSP; no PQ; no coercion resistance.	Factor-m faster ($O(n \times m)$); no trusted counter; SMDC + Kyber-768.
[20]	Hyperledger Besu + (k, ε)-DP + Web3 SSI.	$\sim$1 s/TX; $\geq$ 98% accuracy; 10K–50K votes, 2–8 candidates.	Approximate tally (unsafe in close races); no ZK proofs, anonymity, PQ.	100% exact tally; 70.1 ms/vote; $\sim$200 TPS; coercion + ZK + PQ.
[21]	Hyperledger Fabric + dual-biometric (finger-print+face) + AES-256.	87% biometric enrollment; $\sim$88% successful casts; 100-participant trial.	No homomorphic tally, ZK, anonymity, or coercion resistance.	Paillier tally, ring sigs, SMDC, threshold DJN; 10K-tx Caliper benchmark.
[22]	Paillier + timed-release + partial-knowledge proofs.	Semantic security; tally privacy + temporal fairness; feasibility eval.	Single TRE trust point; no anonymity, coercion resistance, PQ.	Ring sigs, SMDC, threshold DJN, Kyber-768.
[23]	$\Theta(1)$ ballot size; 1-layer liquid democracy; game-based proofs.	$O(n)$ overall; 6$\times$ faster than VoteAgain at 50K; formal proofs.	No homomorphic tally, biometric, SA^2, threshold, PQ, or deployed blockchain.	Seven protocols on Fabric v2.5; SA^2 + threshold + Kyber-768.
[24]	Survey: platform, consensus, crypto, security, deployment.	Identifies gaps in coercion resistance, PQ, scalable tally; most PoC-scale.	(Survey: documents the gaps rather than filling them.)	Fills all flagged gaps: SMDC, Kyber-768, $O(n)$/candidate tally, Caliper-benchmarked Fabric.

Continued on next page

Table 4.10 continued from previous page

Ref.	Methodology	Performance	Their Gaps	Our Edge
This Work	Seven-protocol stack on Fabric v2.5; novel SMDC (k=5), behavioural duress, coercion short-circuit.	70.1 ms/vote; tally 6.1 μs/cand.; $O(n)$/cand., $O(n \cdot m)$ total; $\sim$200 TPS @ 100% (10K-tx).	Biom. is hash-eq. stub (no FAR/FRR, no user trial); bottleneck not isolated.	Only system with all: privacy, coercion, PQ, threshold, ProVerif + game proofs, real Fabric deployment.

The proposed framework demonstrates clear advantages across every dimension that prior work reports. The per-vote pipeline time of 70.1 ms is more than an order of magnitude faster than BP-Vot's $\sim$1 s/TX, and the $\sim$200 TPS sustained Caliper-verified throughput far exceeds the throughput numbers any of the cited works publish. Unlike BP-Vot, which trades exact tallying for differential privacy, the proposed framework reports a 100% exact tally. The $O(n)$-per-candidate, $O(n \times m)$-total complexity is asymptotically superior to ISE-Voting's $O(n \times m^2)$ scaling by a factor of m. Yin et al. achieve coercion resistance with formal game-based proofs, but their scheme lacks homomorphic tallying, post-quantum security, threshold decryption, and a production blockchain integration, all of which the proposed framework provides. Yuan et al.'s timed-release scheme uses Paillier encryption but provides no anonymity, no coercion resistance, and no post-quantum protection. To the best of our knowledge, the proposed framework is the first system to combine seven cryptographic protocols, novel SMDC coercion resistance, behavioural duress detection, and a Caliper-benchmarked Hyperledger Fabric v2.5 deployment in a single architecture.

5. Conclusion and Future Work

5.1 Introduction

In this chapter, we present our concluding thoughts on the proposed framework. We also address the constraints encountered during our work and highlight future directions for further exploration in this area of research.

5.2 Summary of the Study

Our thesis introduced a blockchain-based electronic voting system that integrates seven cryptographic protocols into a unified voting pipeline to provide broad security coverage. The proposed framework addresses key gaps in existing e-voting research by simultaneously achieving ballot privacy, vote validity, voter anonymity, coercion resistance, privacy-preserving aggregation, post-quantum security, and production blockchain integration.

Our framework combines Paillier homomorphic encryption [8] for encrypted tallying, Pedersen commitments [17] with zero-knowledge Σ-proofs (Strong Fiat-Shamir [9]) for vote validity verification, linkable ring signatures [10] for anonymous ballot submission with double-vote detection, SMDC deniable credentials for coercion resistance, SA^2 samplable anonymous aggregation [11] for two-server vote privacy, Kyber768 [12] hybrid encryption for post-quantum protection, and Damgård-Jurik-Nielsen threshold decryption [18] for distributed key management. To the best of our knowledge, no existing system integrates this many cryptographic protocols in a single, functional voting pipeline.

We implemented the system in Go and deployed it on a production Hyperledger Fabric v2.5 [1] blockchain network. Performance evaluation confirmed that the proposed framework achieves $O(n)$-per-candidate tally complexity (and $O(n \cdot m)$ total) with a constant per-vote per-candidate cost of approximately 6.1 μs, verified empirically from 100 to 100 000 voters and across $m \in [2, 50]$ candidates. The complete seven-protocol vote casting pipeline processes a single vote in 70.1 ms (per-phase breakdown sums to 70.08 ms; see Table 4.3). The Hyperledger Fabric network sustains approximately 200 TPS with 100% success rate at 10,000 transactions, as measured by Hyperledger Caliper benchmarks. Post-quantum protection via the Kyber768 hybrid encryption layer adds

only 5.35 ms of overhead per vote (approximately 7.6% of the 70.1 ms pipeline), showing that quantum-resistant security is achievable at modest computational cost.

Our novel SMDC coercion resistance mechanism, with $k = 5$ credential slots (1 real and 4 fake), provides computationally indistinguishable real and fake credentials through Pedersen's hiding property. Under coercion, voters can present fake credentials that produce valid-looking but silently discarded votes, while the real slot index is deterministically re-derivable only by the legitimate voter through HMAC-SHA256. This is the first complete implementation of deniable credentials in a blockchain e-voting context.

Our comparative analysis with state-of-the-art systems confirmed that the proposed framework delivers strong performance while offering broader security coverage. The $O(n)$-per-candidate (and $O(n \cdot m)$-total) tally complexity is a factor-m improvement over ISE-Voting's $O(n \cdot m^2)$, and the 100% exact tallying contrasts with BP-Vot's $\geq$98% differential-privacy approximation. Linear extrapolation of our tally benchmarks projects feasibility for national-scale elections: approximately 5.1 minutes per candidate (and approximately 51 minutes total for $m = 10$ candidates) to tally 50 million voters.

5.3 Limitations and Future Work

Although our proposed framework has shown notable progress in e-voting security, there are some limitations that can be improved in future work.

- **Universally Composable Security Proofs**: While our ProVerif verification confirms security under the symbolic (Dolev-Yao) model, game-based or universally composable (UC) proofs [40] would provide stronger guarantees that security properties are preserved under arbitrary protocol composition. Future work should formalize the 17-step pipeline in the UC framework, following recent advances such as E-cclesia's UC-secure self-tallying approach, and verify that ballot privacy, coercion resistance, and verifiability hold simultaneously against computational adversaries.

- **Full Post-Quantum Migration Roadmap**: ML-KEM-768 [12] currently protects only the integrity binding of vote ciphertexts. A complete post-quantum migration entails three further replacements. (i) The linkable ring signatures rely on the discrete logarithm problem and should be replaced with a lattice-based linkable ring signature (e.g., one constructed from Module-SIS, the assumption underlying ML-DSA in FIPS 204 [14]). (ii) Paillier homomorphic tallying is vulnerable to Shor's algorithm and should be migrated to a fully homomorphic encryption scheme such as BFV, BGV, or CKKS. (iii) Threshold decryption should likewise be reformulated over the chosen lattice-based scheme. A staged migration is realistic: signature replacement first (ML-DSA is already standardised), homomorphic-tally replacement second (lattice FHE has acceptable performance for the small plaintext domain $\{0, 1\}$ used here), and threshold-decryption replacement last. Until that migration is complete, the

present hybrid construction provides confidentiality against harvest-now-decrypt-later adversaries, which is the most pressing of the post-quantum threats for ballots that must remain secret for decades.

- **Single Orderer Limitation**: Our current deployment uses a single Raft orderer node, which provides crash fault tolerance but not Byzantine fault tolerance. For high-stakes elections where orderer nodes may be adversarial, migrating to a BFT consensus mechanism (e.g., SmartBFT in Fabric v3.x) would strengthen the system's resilience.

- **Fixed Ring Size**: The ring size is fixed at $|\mathcal{R}| = 100$ in our implementation. While this provides strong anonymity — the signer is statistically indistinguishable among the 100 ring members under DDH — a dynamic ring size mechanism that adapts to the voter pool size and latency requirements would provide more flexible deployment options.

- **Blockchain Scalability at Extreme Load**: Our Hyperledger Caliper benchmarks revealed that the Fabric Gateway's default concurrency limit (500 connections) causes failures at transaction volumes above 25,000. Production deployments for national-scale elections would require careful infrastructure engineering including connection pool tuning, horizontal peer scaling, and load balancing.

- **Mobile Client Implementation**: Our current implementation provides a server-side API but lacks a dedicated mobile client application. For real-world deployment, a mobile application with on-device biometric authentication, secure enclave key storage, and offline credential management would improve voter accessibility.

- **Threshold Parameter Optimization**: The current threshold Paillier key generation takes approximately 30 seconds due to safe prime generation for the DJN scheme. For elections with frequent key rotation, exploring alternative threshold schemes (e.g., based on class groups) could reduce setup time.

- **Large-Scale User Study**: Our system has been evaluated through automated benchmarks and unit tests but has not undergone a large-scale user study with real voters. A deployment trial with hundreds of participants would provide valuable data on usability, biometric enrollment success rates, and voter confidence.

The limitations we identify above and the future work directions suggest a roadmap for advancing our framework toward production-ready deployment. By addressing formal verification, full post-quantum migration, infrastructure scaling, and user experience design, the system can evolve into a practical solution for secure, coercion-resistant democratic elections.

In summary, the proposed framework achieves the following key quantitative results:

1. **6.1** μ**s** per-vote per-candidate tally cost, constant across 100 to 100 000 voters and across $m \in [2, 50]$ candidates ($O(n)$ per candidate, $O(n \cdot m)$ total).
2. **70.1 ms** complete seven-protocol pipeline time per vote (sum of per-phase Table 4.3).
3. $\sim$**200 TPS** sustained blockchain throughput with 100% success at 10,000 transactions.
4. **5.35 ms** Kyber768 hybrid-encryption overhead per vote ($\approx$7.6% of pipeline) for post-quantum protection.
5. **7 cryptographic protocols** integrated in a single pipeline, more than any existing system.
6. **188 tests** (159 unit/integration + 29 benchmarks) with 100% pass rate and 78–92% coverage on core packages.

In summary, we believe our study presents a promising and rigorous approach to designing next-generation blockchain-based e-voting systems. the proposed framework is, to the best of our knowledge, the first system to simultaneously provide ballot privacy, vote validity, voter anonymity, coercion resistance, privacy-preserving aggregation, a post-quantum encryption layer, and production blockchain integration with thorough empirical validation. While the encryption layer achieves post-quantum protection via Kyber768 (ML-KEM-768, NIST FIPS 203), the ring signature and homomorphic tallying components remain classically secure, with lattice-based replacements identified as future work. We believe that the design principles, cryptographic constructions, and engineering practices presented in this thesis can serve as a foundation for future research in this critical area of democratic technology.

Reference

[1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*, pp. 1–15, ACM, 2018.

[2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system." https://bitcoin.org/bitcoin.pdf, 2008. Original blockchain whitepaper.

[3] M. A. Specter, J. Koppel, and D. Weitzner, "The ballot is busted before the blockchain: A security analysis of Voatz, the first internet voting application used in U.S. federal elections," in *29th USENIX Security Symposium*, pp. 1535–1553, USENIX Association, 2020.

[4] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," Tech. Rep. FIPS PUB 197, NIST, 2001.

[5] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[6] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," 2014. Ethereum Project Yellow Paper.

[7] Y. Ucbas, A. Eleyan, M. Hammoudeh, and M. Alohaly, "Performance and scalability analysis of Ethereum and Hyperledger Fabric," *IEEE Access*, vol. 11, pp. 67156–67167, 2023.

[8] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Advances in Cryptology — EUROCRYPT '99*, vol. 1592 of *Lecture Notes in Computer Science*, pp. 223–238, Springer, 1999.

[9] D. Bernhard, O. Pereira, and B. Warinschi, "How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios," in *Advances in Cryptology —*

ASIACRYPT 2012, vol. 7658 of *Lecture Notes in Computer Science*, pp. 626–643, Springer, 2012.

[10] J. K. Liu, V. K. Wei, and D. S. Wong, "Linkable spontaneous anonymous group signature for ad hoc groups," in *Information Security and Privacy — ACISP 2004*, vol. 3108 of *Lecture Notes in Computer Science*, pp. 325–335, Springer, 2004.

[11] K. Talwar, S. Wang, A. McMillan, V. Jina, V. Feldman, B. Basile, A. Cahill, Y. S. Chan, M. Chatzidakis, O. Chick, M. Chitnis, S. Ganta, Y. Goren, F. Granqvist, K. Guo, F. Jacobs, O. Javidbakht, A. Liu, R. Low, D. Mascenik, S. Myers, D. Park, W. Park, G. Parsa, T. Pauly, C. Priebe, R. Rishi, G. Rothblum, M. Scaria, L. Song, C. Song, K. Tarbe, S. Vogt, L. Winstrom, and S. Zhou, "Samplable anonymous aggregation for private federated data analysis," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, ACM, 2024.

[12] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard (FIPS 203)," Tech. Rep. FIPS 203, U.S. Department of Commerce, Aug. 2024. Standardizes ML-KEM (formerly CRYSTALS-Kyber).

[13] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[14] National Institute of Standards and Technology, "Module-lattice-based digital signature standard (FIPS 204)," Tech. Rep. FIPS 204, U.S. Department of Commerce, Aug. 2024. Standardizes ML-DSA (formerly CRYSTALS-Dilithium).

[15] A. Juels, D. Catalano, and M. Jakobsson, "Coercion-resistant electronic elections," in *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES '05)*, pp. 61–70, ACM, 2005.

[16] M. R. Clarkson, S. Chong, and A. C. Myers, "Civitas: Toward a secure voting system," in *2008 IEEE Symposium on Security and Privacy*, pp. 354–368, IEEE, 2008.

[17] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology — CRYPTO '91*, vol. 576 of *Lecture Notes in Computer Science*, pp. 129–140, Springer, 1991.

[18] I. Damgård, M. Jurik, and J. B. Nielsen, "A generalization of Paillier's public-key system with applications to electronic voting," *International Journal of Information Security*, vol. 9, no. 6, pp. 371–385, 2010.

[19] J. Zhang, C. Wu, R. S. Sherratt, and J. Wang, "An improved secure and efficient E-Voting scheme based on blockchain systems," *IEEE Internet of Things Journal*, vol. 12, no. 7, pp. 8628–8640, 2025.

[20] H. Baniata and G. Caluna, "BP-Vot: Blockchain-based e-voting using smart contracts, differential privacy and self-sovereign identities," *IEEE Access*, vol. 13, pp. 46106–46123, 2025.

[21] M. J. H. Faruk, F. Alam, M. Islam, and A. Rahman, "Transforming online voting: A novel system utilizing blockchain and biometric verification for enhanced security, privacy, and transparency," *Cluster Computing*, vol. 27, no. 4, pp. 4015–4034, 2024.

[22] K. Yuan, P. Sang, J. Ge, B. Zhou, and C. Jia, "A timed-release E-Voting scheme based on Paillier homomorphic encryption," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 5, pp. 740–753, 2024.

[23] Z. Yin, B. Zhang, A. Nastenko, R. Oliynykov, and K. Ren, "A scalable coercion-resistant voting scheme for blockchain decision-making," *IEEE Transactions on Dependable and Secure Computing*, 2024. Also available as IACR ePrint 2023/1578.

[24] H. O. Ohize, A. J. Onumanyi, B. U. Umar, L. A. Ajao, R. O. Isah, E. M. Dogo, B. K. Nuhu, O. M. Olaniyi, J. G. Ambafi, V. B. Sheidu, and M. M. Ibrahim, "Blockchain for securing electronic voting systems: A survey of architectures, trends, solutions, and challenges," *Cluster Computing*, 2024.

[25] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[26] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology — CRYPTO '86*, vol. 263 of *Lecture Notes in Computer Science*, pp. 186–194, Springer, 1986.

[27] R. L. Rivest, A. Shamir, and Y. Tauman, "How to leak a secret," in *Advances in Cryptology — ASIACRYPT 2001*, vol. 2248 of *Lecture Notes in Computer Science*, pp. 552–565, Springer, 2001.

[28] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[29] National Institute of Standards and Technology, "Secure hash standard (SHS)," Tech. Rep. FIPS PUB 180-4, National Institute of Standards and Technology, 2015.

[30] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," Tech. Rep. FIPS PUB 202, National Institute of Standards and Technology, 2015.

[31] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication." RFC 2104, Internet Engineering Task Force, 1997.

[32] National Institute of Standards and Technology, "The keyed-hash message authentication code (HMAC)," Tech. Rep. FIPS PUB 198-1, National Institute of Standards and Technology, 2008.

[33] M. J. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," Tech. Rep. NIST SP 800-38D, National Institute of Standards and Technology, 2007.

[34] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, pp. 1175–1191, ACM, 2017.

[35] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)," in *USENIX Annual Technical Conference (ATC)*, pp. 305–319, 2014.

[36] Hyperledger Foundation, "Hyperledger Caliper: Blockchain performance benchmark framework." https://hyperledger.github.io/caliper/, 2024. v0.6.0.

[37] Cloudflare, Inc., "CIRCL: Cloudflare interoperable reusable cryptographic library." https://github.com/cloudflare/circl, 2024. Go library implementing post-quantum cryptographic primitives including ML-KEM (Kyber).

[38] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

[39] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[40] L. Ackermann, M. Arapinis, P. Georgiou, N. Lamprou, L. Mareková, and T. Zacharias, "E-cclesia: Universally composable self-tallying elections over anonymous broadcast," *IACR Communications in Cryptology*, vol. 2, Oct. 2025. Earlier version: IACR ePrint 2020/513.

A. ProVerif Formal Verification Artefacts

This appendix reproduces the ProVerif source models and the corresponding tool output transcripts that underpin the formal verification results summarised in Table 4.9 of Chapter 4. The three models target ballot privacy, individual verifiability, and voter eligibility, respectively; each is verified by ProVerif 2.05 [38] under the symbolic Dolev–Yao adversary model. All three queries return `true`.

The model files are versioned in the project repository under the directory `proverif/`. The shell script `proverif/run_verification.sh` reproduces all three runs in sequence and prints the `RESULT` lines.

A.1 Reproducing the Verification

Install ProVerif via OPAM (OCaml package manager):

```
opam install proverif
```

Run all three verifications:

```
cd proverif
./run_verification.sh
```

The expected condensed output is:

```
RESULT Observational equivalence is true.
RESULT inj-event(BallotRecorded(b_2)) ==>
       inj-event(VoterCastBallot(pkV,b_2)) is true.
RESULT event(ValidBallotCast(pkV)) ==>
       event(VoterRegistered(pkV)) is true.
```

A.2 Ballot Privacy: `privacy.pv`

Ballot privacy is verified via observational equivalence: an adversary controlling the network cannot distinguish between two honest voters swapping their ballots. The model uses the `diff[v0, v1]` construct to test whether the protocol is observationally equivalent under both ballot assignments.

Source: proverif/privacy.pv

```
(* CovertVote: Ballot Privacy via Observational Equivalence *)
(* NO queries allowed in this file -- diff mode only *)

type skey.
type pkey.
type vote.
type nonce.
type commitment.
type zkprf.
type signature.
type keyimage.

free ch: channel.
free bb: channel.
free sa2_a: channel [private].
free sa2_b: channel [private].

free v0: vote.
free v1: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.
fun commit(vote, nonce): commitment.
fun zkp_make(vote, nonce, commitment): zkprf.
fun ring_sign(bitstring, skey, pkey): signature.
fun key_image(skey): keyimage.
fun sa2_share_a(bitstring, nonce): bitstring.
fun sa2_share_b(bitstring, nonce): bitstring.
fun make_ballot(bitstring, commitment, zkprf, signature, keyimage): bitstring.

let SA2ServerA() = in(sa2_a, x: bitstring); 0.
let SA2ServerB() = in(sa2_b, x: bitstring); 0.

let VoterPrivacy(skV: skey, pkEA: pkey) =
    new r_enc: nonce;
    new r_com: nonce;
    new r_sa2: nonce;
    let v = diff[v0, v1] in
    let encVote = enc(v, pkEA, r_enc) in
    let com = commit(v, r_com) in
    let zk = zkp_make(v, r_com, com) in
    let sig = ring_sign(encVote, skV, pk(skV)) in
    let ki = key_image(skV) in
    let b = make_ballot(encVote, com, zk, sig, ki) in
    out(sa2_a, sa2_share_a(encVote, r_sa2));
    out(sa2_b, sa2_share_b(encVote, r_sa2));
    out(bb, b);
    0.

process
    new skEA: skey;
    let pkEA = pk(skEA) in
    out(ch, pkEA);
    (
        new skAlice: skey;
        new skBob: skey;
        out(ch, pk(skAlice));
```

```
        out(ch, pk(skBob));
        VoterPrivacy(skAlice, pkEA) |
        VoterPrivacy(skBob, pkEA) |
        SA2ServerA() |
        SA2ServerB()
    )
```

ProVerif output (excerpt)

```
$ proverif privacy.pv
...
RESULT Observational equivalence is true.
--------------------------------------------------------------
Verification summary:

Observational equivalence is true.

--------------------------------------------------------------
```

Interpretation. The "`Observational equivalence is true`" result means that for every possible adversary strategy, the trace of network observations is identical regardless of whether each voter cast `v0` or `v1`. Therefore an adversary cannot infer how any individual voter voted from network-level observations alone.

A.3 Individual Verifiability: `verifiability.pv`

Individual verifiability is verified via injective correspondence: every recorded ballot on the bulletin board corresponds to exactly one authentic cast event by a registered voter. The query asserts `BallotRecorded(b)` $\Longrightarrow$ `inj-event(VoterCastBallot(pkV, b))`.

Source: `proverif/verifiability.pv`

```
(* CovertVote: Individual Verifiability *)

type skey.
type pkey.
type vote.
type nonce.
type commitment.
type zkprf.
type signature.
type keyimage.

free ch: channel.
free bb: channel.
free auth_bb: channel [private].
free sa2_a: channel [private].
free sa2_b: channel [private].

free v0: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.
```

```
fun commit(vote, nonce): commitment.
fun zkp_make(vote, nonce, commitment): zkprf.
fun ring_sign(bitstring, skey, pkey): signature.
fun key_image(skey): keyimage.
fun sa2_share_a(bitstring, nonce): bitstring.
fun sa2_share_b(bitstring, nonce): bitstring.
fun make_ballot(bitstring, commitment, zkprf, signature, keyimage): bitstring.

event VoterCastBallot(pkey, bitstring).
event BallotRecorded(bitstring).

query b: bitstring, pkV: pkey;
    event(BallotRecorded(b)) ==> inj-event(VoterCastBallot(pkV, b)).

let SA2ServerA() = in(sa2_a, x: bitstring); 0.
let SA2ServerB() = in(sa2_b, x: bitstring); 0.

let VoterVerifiable(skV: skey, v: vote, pkEA: pkey) =
    new r_enc: nonce;
    new r_com: nonce;
    new r_sa2: nonce;
    let encVote = enc(v, pkEA, r_enc) in
    let com = commit(v, r_com) in
    let zk = zkp_make(v, r_com, com) in
    let sig = ring_sign(encVote, skV, pk(skV)) in
    let ki = key_image(skV) in
    let b = make_ballot(encVote, com, zk, sig, ki) in
    event VoterCastBallot(pk(skV), b);
    out(sa2_a, sa2_share_a(encVote, r_sa2));
    out(sa2_b, sa2_share_b(encVote, r_sa2));
    out(auth_bb, b);
    0.

let BulletinBoard() =
    in(auth_bb, b: bitstring);
    event BallotRecorded(b);
    out(bb, b);
    0.

process
    new skEA: skey;
    let pkEA = pk(skEA) in
    out(ch, pkEA);
    (
        new skV: skey;
        VoterVerifiable(skV, v0, pkEA) |
        BulletinBoard() |
        SA2ServerA() |
        SA2ServerB()
    )
```

ProVerif output (excerpt)

```
$ proverif verifiability.pv
...
Starting query inj-event(BallotRecorded(b_2))
              ==> inj-event(VoterCastBallot(pkV,b_2))
goal reachable: b-inj-event(VoterCastBallot(...))
              -> inj-event(BallotRecorded(...))
```

```
The hypothesis occurs strictly before the conclusion.

RESULT inj-event(BallotRecorded(b_2)) ==>
       inj-event(VoterCastBallot(pkV,b_2)) is true.
--------------------------------------------------------------
Verification summary:

Query inj-event(BallotRecorded(b_2)) ==>
      inj-event(VoterCastBallot(pkV,b_2)) is true.

--------------------------------------------------------------
```

Interpretation. The injective correspondence guarantees that no recorded ballot is forged or duplicated: each `BallotRecorded` event on the bulletin board is matched to a unique prior `VoterCastBallot` event by an authenticated voter, providing individual verifiability.

A.4 Voter Eligibility: `eligibility.pv`

Voter eligibility is verified via causal correspondence: every valid ballot cast was preceded by a legitimate registration event with the registration authority. The query asserts `ValidBallotCast(pkV)` $\Longrightarrow$ `VoterRegistered(pkV)`.

Source: `proverif/eligibility.pv`

```
(* CovertVote: Voter Eligibility *)
(* NO diff allowed -- query mode only *)

type skey.
type pkey.
type vote.
type credential.
type nonce.

free ch: channel.
free bb: channel.
free reg: channel [private].

free v0: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.
fun real_cred(skey): credential.
fun fake_cred(skey): credential.

event VoterRegistered(pkey).
event ValidBallotCast(pkey).

query pkV: pkey;
    event(ValidBallotCast(pkV)) ==> event(VoterRegistered(pkV)).

let RegistrationAuthority() =
    in(reg, voterPK: pkey);
    new skCred: skey;
```

```
    out(reg, (real_cred(skCred), fake_cred(skCred)));
    0.

let VoterEligible(skV: skey, v: vote, pkEA: pkey) =
    out(reg, pk(skV));
    in(reg, (realC: credential, fakeC: credential));
    event VoterRegistered(pk(skV));
    new r_enc: nonce;
    let encVote = enc(v, pkEA, r_enc) in
    event ValidBallotCast(pk(skV));
    out(bb, encVote);
    0.

process
    new skEA: skey;
    let pkEA = pk(skEA) in
    out(ch, pkEA);
    (
        new skV: skey;
        RegistrationAuthority() |
        VoterEligible(skV, v0, pkEA)
    )
```

ProVerif output (excerpt)

```
$ proverif eligibility.pv
...
Starting query event(ValidBallotCast(pkV))
               ==> event(VoterRegistered(pkV))
goal reachable: b-event(VoterRegistered(pk(skV[])))
               && mess(reg[],voterPK_1)
               -> event(ValidBallotCast(pk(skV[])))

RESULT event(ValidBallotCast(pkV)) ==>
       event(VoterRegistered(pkV)) is true.
------------------------------------------------------------
Verification summary:

Query event(ValidBallotCast(pkV)) ==>
      event(VoterRegistered(pkV)) is true.

------------------------------------------------------------
```

Interpretation. The causal correspondence guarantees that any ballot accepted as "valid" is necessarily preceded by a successful registration of the same voter public key with the registration authority. This rules out unauthorised vote injection by unregistered parties.

A.5 Summary

Table 4.9 of Chapter 4 summarises the three results in tabular form: all three properties hold under the Dolev–Yao adversary model. Together they establish symbolic-model security against eavesdropping, replay, and man-in-the-middle attacks at the protocol level.

Stronger guarantees in the computational model (game-based or universally composable proofs) are identified as future work in Chapter 5.

Please use the following page format for the cover of your report.

A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security

by

Monowar Hossen Sourav
221002578

Md. Sajeeb Mia | **Md. Naim Ahammed**
221002251 | **221002599**

Thesis for the degree of

Bachelor of Science in Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GREEN UNIVERSITY OF BANGLADESH

SPRING 2026